

Integrating Heterogeneous Environmental Permitting Spreadsheets through LLM-Assisted Schema Matching and Deterministic Normalization

by

Dorna Dehghani

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF

MASTER OF SCIENCE

in

The Faculty of Graduate and Postdoctoral Studies

(Computer Science)

THE UNIVERSITY OF BRITISH COLUMBIA

(Vancouver)

April 2026

© Dorna Dehghani 2026

The following individuals certify that they have read, and recommend to the Faculty of Graduate and Postdoctoral Studies for acceptance, the thesis entitled:

Integrating Heterogeneous Environmental Permitting Spreadsheets through LLM-Assisted Schema Matching and Deterministic Normalization

submitted by **Dorna Dehghani** in partial fulfillment of the requirements for the degree of **Master of Science** in **Computer Science**.

Examining Committee:

Laks V.S. Lakshmanan, Professor, Computer Science, UBC
Supervisor

Raymond Ng, Professor, Computer Science, UBC
Supervisory Committee Member

Abstract

Environmental assessment and permitting processes generate large volumes of administrative data, typically stored in project-specific spreadsheets that record iterative comment–response exchanges across multiple review rounds. Although these spreadsheets contain semantically similar information—such as comment text, response text, dates, agencies, and review status—their schemas, layouts, and value encodings vary across projects. This structural heterogeneity prevents systematic cross-project analysis and limits transparency, reproducibility, and scalability.

This thesis formulates the integration of heterogeneous permitting spreadsheets as a schema matching and normalization problem and presents a deterministic end-to-end pipeline that transforms project-level Excel files into a unified long-format dataset. The pipeline combines column-level schema alignment with value-level normalization. Instruction-tuned large language models (LLMs) generate enriched column descriptions and support interpretable semantic equivalence decisions within a structured matching framework, while preserving auditability by serving as contextual representations rather than autonomous transformation logic. Following schema alignment, rule-based transformations standardize dates, agency names, compound identifiers, null placeholders, and wide-format review rounds, producing a consistent representation in which each record corresponds to a single comment–response pair with associated metadata.

Evaluation is conducted using a project-level train–test split to assess generalization to previously unseen spreadsheet structures. Results indicate that LLM-assisted semantic matching combined with deterministic preprocessing enables robust schema alignment across heterogeneous files while maintaining interpretability and reproducibility. Question answering over the normalized dataset further demonstrates that integration reduces analytical effort for cross-project queries.

This work provides a practical and reproducible framework for structured

Abstract

data integration in regulatory and administrative contexts under local, single-GPU deployment constraints.

Lay Summary

Environmental assessment and mine permitting processes generate large numbers of spreadsheets that record comments and responses between government agencies and project proponents. Although these spreadsheets contain similar types of information, they are structured differently from one project to another. This makes it difficult to combine them, compare projects, or answer simple questions without significant manual effort.

This thesis develops a systematic method to transform these inconsistent spreadsheets into a single, standardized dataset. The approach uses large language models to help identify which columns correspond to the same type of information, and applies clear, rule-based steps to clean and reorganize the data. The final result is a unified format where each comment and response can be analyzed consistently across projects.

By making this data easier to integrate and query, the work supports more transparent, reproducible, and efficient analysis in regulatory and administrative contexts.

Preface

All work presented in this thesis is the original work of the author and was conducted under the supervision of Prof. Laks V.S. Lakshmanan.

The spreadsheet datasets used in this research were provided by Sean Shaw, Ph.D., P.Geo., Executive Director & Deputy Chief Permitting Officer, Major Mines Office, for research purposes. The data consists of non-public administrative permitting records. The research was conducted in compliance with the data-sharing conditions communicated at the time of access, and no external LLM services were used to process confidential data.

In the preparation of this manuscript, Geminin and ChatGPT were used to refine the clarity, grammar, and formatting of the text based on the author's original drafts and technical findings. All intellectual content, methodology design, experimental execution, and final interpretations remain the sole responsibility of the author.

Table of Contents

Abstract	iii
Lay Summary	v
Preface	vi
Table of Contents	vii
List of Tables	xi
List of Figures	xiii
List of Abbreviations	xiv
Acknowledgements	xv
Dedication	xvi
1 Introduction	1
1.1 Motivation	1
1.2 Contributions	3
1.3 Thesis Outline	4
2 Problem Statement	5
2.1 Context and Motivation	5
2.2 Limits of Current Spreadsheet Schemas	5
2.3 Real-World Data Issues and Consequences	6
2.4 Problem Statement	7
2.5 Research Questions	7
2.6 Motivation for This Work	8
2.7 How Language Models Can Help Schema Matching	8
2.8 Generalization of Rule-Based Normalization	8

Table of Contents

2.9	Does Normalization Enable Question Answering?	9
2.10	Scope and Assumptions	9
3	Related Works	10
3.1	Classical Schema Matching Approaches	10
3.1.1	SMAT Architectural Foundations and Contributions	11
3.2	Large Language Models for Schema Matching and Data Preparation	12
3.2.1	Philosophical and Technical Shifts Toward LLMs	12
3.2.2	Jellyfish: Specialized LLM Framework for Preprocessing	12
3.3	Scaling Schema Matching: Scoping, Retrieval, and Reranking Pipelines	14
3.3.1	Scoping, Configuration, and Human-in-the-Loop Perspectives	14
3.3.2	The Magneto Hybrid Framework	15
3.3.3	LLMatch and the SchemaNet Benchmark	15
3.4	Question Answering on Structured Data	16
3.4.1	Benchmarks for Complex Analytical Reasoning	16
3.4.2	Table Question Answering (TQA) Paradigms	16
3.5	LLMs and Spreadsheet Understanding	17
3.5.1	SpreadsheetLLM and SheetCompressor Framework	17
3.6	Local LLM Research: Infrastructure and Model Specifics	17
3.6.1	The Role of the Ollama Framework	18
3.6.2	Mistral 7B: Efficiency and Architecture	18
4	Data	19
4.1	Data Sources and General Characteristics	19
4.2	Structural Heterogeneity and Integration Challenges	20
4.2.1	Column Naming and Definition Inconsistencies	20
4.2.2	Representation of Review Rounds: Wide vs. Long Formats	22
4.3	Motivation for Normalization	23
4.4	Train and Test Data Separation	24
4.4.1	Definition of Training and Test Sets	24
4.4.2	Specification of Training and Test Files	25
4.4.3	Rationale for Project-Level Splitting	29
4.4.4	Role of the Train-Test Split in the Pipeline	30
4.4.5	Limitations	31

Table of Contents

5	Methodology	32
5.1	Input Identification and Structural Cleaning	33
5.1.1	Sheet and Header Detection	33
5.1.2	Jellyfish Input Preparation	34
5.2	Column-Level Schema Matching	35
5.2.1	Motivation and Problem Definition	35
5.2.2	Target Schema Design	35
5.2.3	Column Description Generation	35
5.2.4	Jellyfish Schema Matching	40
5.2.5	Handling Ambiguity Between Similar Fields	43
5.2.6	Mapping Selection and Validation	44
5.2.7	Design Rationale and Limitations	44
5.3	Value-Level Processing and Rule-Based Transformations	45
5.3.1	Normalization Procedure	45
5.3.2	Handling of Review Rounds and Wide-to-Long Normalization	45
5.3.3	Target Schema and Output Representation	46
5.3.4	Example: Wide-to-Long Normalization	46
5.3.5	Rationale and Limitations	47
5.4	Downstream Task: Question Answering	47
5.4.1	Task Setup	48
5.4.2	Query Execution via Code Generation	48
5.4.3	Model Selection	49
5.4.4	Evaluation Strategy	50
5.4.5	Scope and Limitations	50
6	Experiments and results	51
6.1	Overview of Evaluation Setup	51
6.2	Evaluation Metrics and Their Limitations	53
6.3	Schema Matching Results	54
6.3.1	Training Set Results	54
6.3.2	Test Set Results	59
6.4	Aggregated Project-Level Results	67
6.5	Value Matching: Qualitative Error Analysis	68
6.6	Question Answering: Results and Error Analysis	69
6.7	Summary of Key Findings	76
6.8	Reflection on Strategic Objectives	77
6.9	Open Questions and Unexpected Behaviors	78
7	Conclusion and Future Work	79

Table of Contents

7.1	Limitations	80
7.2	Future Work	81
	Bibliography	83
 Appendices		
A	Representative Spreadsheet Heterogeneity	87
A.1	Long-Format Example	87
A.2	Wide-Format Example	88
B	Prompt Templates Used in This Thesis	90
B.1	Description Generation Prompts	90
B.2	Jellyfish Schema Matching Prompts	91

List of Tables

3.1	Comparison of Traditional Schema Matchers and SMAT . . .	12
3.2	Comparison of Jellyfish (LLM) and SMAT (DL) across key model components	13
4.1	Example of a long-format representation of a multi-round re- view conversation. Each row corresponds to a single comment- response pair within the same conversation.	22
4.2	Example of a wide-format representation of a multi-round re- view conversation. All review rounds for a single conversation are stored in one row using round-specific columns.	22
4.3	Spreadsheet sources used for the training subset.	27
4.4	Spreadsheet sources used for the test subset.	29
5.1	Target schema used for normalization, with field-level seman- tic definitions.	36
5.2	Example outputs for one source column (Date Received) generated by four candidate LLMs using identical prompts (column name + sample values), together with per-call gen- eration time.	39
5.3	Example of the normalized target schema. Each record repre- sents a single comment-response pair, regardless of the orig- inal source format.	46
6.1	Per-project evaluation results on the training set using one description per source and target column.	56
6.2	Per-project evaluation results on the training set using three descriptions per selected source-target candidate pairs with majority voting.	58
6.3	Per-project evaluation results on the test set using one de- scription per source and target column.	62

List of Tables

6.4	Per-project evaluation results on the test set using three descriptions per selected source–target candidate pairs with majority voting.	66
6.5	Final Result Correctness across Training and Test sets.	67
6.6	Average Column Detection and Structure Detection Accuracy.	68
6.7	QA accuracy and runtime summary on the normalized dataframe using PandasAI. Average and median runtime are measured per question.	70
6.8	Per-question QA outputs for <code>qwen2.5-coder:7b</code> . “Reference” lists the deterministic pandas answer for train/test (when they differ). “ERROR” denotes execution failures or missing code generation.	72
6.9	Per-question QA outputs for <code>deepseek-coder:6.7b</code> . “Reference” lists the deterministic pandas answer for train/test (when they differ). “ERROR” denotes execution failures or missing code generation.	74
B.1	Prompting strategies for column description generation. P=Prompt, FS=Few Shots.	91
B.2	Complete Jellyfish outputs for schema matching under different model sizes (7B, 8B) and prompting strategies (with and without chain-of-thought). The evaluated source–target attribute pair is Round 1: MDRC Member Comment and <code>response_text</code> , for which the correct decision is No	92

List of Figures

- 5.1 Flowchart of the modular normalization pipeline, from raw
Excel inputs to a unified long-format dataset. The dashed
arrow indicates the optional downstream QA validation stage. 33

- A.1 Snapshot of project “2022 12 23 Blackwater_IT” illustrating
a long-format structure. Note how the same Comment ID
appears in multiple rows to represent different review rounds. 87
- A.2 NewAfton C-Zone (Part 1 of 3): Initial columns and Round
1 metadata. 88
- A.3 NewAfton C-Zone (Part 2 of 3): Expansion into Round 2
comments and responses. 88
- A.4 NewAfton C-Zone (Part 3 of 3): Final columns including
Round 3 and project status. 89

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
DL	Deep Learning
DNN	Deep Neural Network
EA	Environmental Assessment
EMLI	Ministry of Energy, Mines and Low Carbon Innovation
ENV	Ministry of Environment and Climate Change Strategy
GPU	Graphics Processing Unit
GQA	Grouped-Query Attention
LLM	Large Language Model
ML	Machine Learning
QA	Question Answering
REPL	Read-Eval-Print-Loop
RL	Reinforcement Learning
SLM	Small Language Model
SMAT	Schema Matching via Transformers
SWA	Sliding Window Attention
TQA	Table Question Answering
HITL	Human in the Loop

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Prof. Laks V.S. Lakshmanan. His guidance, mentorship, and unwavering support were essential throughout my master’s journey. He was more than a research supervisor to me. During challenging times, he showed empathy, patience, and understanding. His encouragement helped me believe in my own abilities, often more than I believed in myself. I could not have completed this degree without his guidance, kindness, and trust.

This research was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) through the Alliance Grant program. I also acknowledge the British Columbia Ministry of Energy, Mines and Low Carbon Innovation (EMLI) for the valuable research discussions and for providing the datasets essential to this work.

I am profoundly thankful to my partner and closest friend, Sadra. He stood by me at every stage of this journey. His patience, calm perspective, and constant support helped me navigate moments of stress and uncertainty. He faced every challenge alongside me and reminded me of strengths I could not always see on my own.

I would also like to thank my friend and roommate, Nasim, who was always willing to listen when I felt anxious about my thesis. Her thoughtful feedback and steady presence brought both clarity and comfort. She helped me keep my spirit and sense of humor alive during demanding periods.

Finally, I owe everything to my parents. Their unconditional love, sacrifices, and continuous support made this achievement possible. They did everything they could to protect me from stress and hardship. I could not have asked for better parents.

Dedication

To Woman,

To Life,

To Freedom.

Chapter 1

Introduction

1.1 Motivation

Environmental assessment (EA) is a regulatory process used to evaluate the potential environmental, social, and economic effects of proposed projects before approval and construction. In practice, EA is not a single decision point but an iterative workflow that involves evidence review, technical feedback, revisions, and follow-up questions across multiple stakeholders and over extended time periods[1].

In the BC mine permitting context, project proponents submit technical materials (e.g., design documents, monitoring plans, impact assessments), and reviewers from multiple organizations examine these submissions. These organizations include provincial government agencies such as the Ministry of Energy, Mines and Low Carbon Innovation (EMLI) and the Ministry of Environment and Climate Change Strategy (ENV), Indigenous Nations participating in the review process, and, in some cases, federal regulators or other technical advisory bodies.

Their role is to assess the sufficiency, consistency, and regulatory compliance of submitted information, raise concerns, request clarifications, and document conditions or follow-up actions where needed. A substantial portion of this review process is recorded as repeated *comment–response exchanges* between reviewers and proponents, often spanning multiple rounds.

In operational workflows, these comment–response exchanges are frequently tracked in spreadsheets. Spreadsheets offer accessibility and flexibility, making them practical for project-level coordination and for capturing evolving information over time. However, spreadsheets are typically designed for local use rather than for cross-project integration and analysis. As a result, even when different project files contain largely the same semantic information (e.g., comment text, response text, dates, authors/agencies, review status, referenced document sections), they often encode it using different

schemas: column names differ across files, round structure may be expressed in incompatible layouts, and values may be encoded inconsistently.

To illustrate this challenge concretely, consider the representation of project actors and identifiers. In one project spreadsheet, an agency might be explicitly labeled in a dedicated “Agency” column, whereas in another, this information is collapsed into a compound identifier such as “ENV-01”. Here, the prefix “ENV” implicitly denotes the Ministry of Environment, while the numeric suffix represents the specific comment index. Such structural variation implies that a simple horizontal concatenation of files is insufficient; a semantic-aware normalization is required to decouple these atomic attributes for cross-project indexing.

This heterogeneity creates a major obstacle to systematic analysis. Without integration, answering even basic questions across projects can require substantial manual effort. More importantly, longitudinal and comparative analyses—such as identifying recurring concerns, measuring response delays across rounds, or comparing review dynamics across agencies—become difficult or infeasible at scale. A unified, structured representation of review conversations would enable consistent aggregation and querying, support transparent reporting, and reduce analytical friction.

An additional practical consideration is deployment feasibility. Many recent data-centric workflows rely on large, proprietary cloud-based models that require substantial computational resources or external API access. In contrast, this thesis emphasizes approaches that can be implemented using lightweight language models deployable on a single GPU. This design choice supports reproducibility, cost-efficiency, and institutional data governance requirements, particularly in regulatory contexts where sensitive project information cannot be transmitted to external services.

Finally, a structured dataset also enables downstream tools for usability and exploration. In particular, question answering (QA) over a normalized dataset can serve as a practical validation: if a dataset has been normalized correctly, natural-language questions about the review process should translate into stable, executable operations over the data. This provides an additional lens for evaluating whether normalization meaningfully improves access to the underlying information, beyond purely schema-level measures.

1.2 Contributions

This thesis addresses the data integration challenge in real-world EA review spreadsheets by formulating it as a schema matching and normalization problem and developing an end-to-end pipeline that produces a unified, analyzable dataset.

The main contributions are:

1. **A deterministic normalization pipeline for heterogeneous review spreadsheets.** The pipeline transforms project-specific Excel files into a single long-format target schema where each record represents one comment–response pair for a specific review round, along with standardized metadata (e.g., project identifier, agency, dates, status, and traceability fields such as source file).
2. **Schema-level matching with LLM-assisted semantic inference.** To map source columns to a unified target schema without labeled training data, the thesis uses an instruction-tuned LLM-based matcher (Jellyfish) that outputs interpretable Yes/No decisions for semantic equivalence between column descriptions. To support this matching, the pipeline generates richer textual descriptions for both source and target columns, providing additional context beyond short headers.
3. **Value-level processing via rule-based transformations.** After schema alignment, the pipeline applies deterministic value-level transformations to standardize agency names, normalize dates, decompose compound identifiers, handle null placeholders, propagate identifiers across multi-row conversations, and convert wide-format round layouts into long-format records.
4. **Downstream validation through question answering over the unified dataset.** The thesis evaluates the usability of the normalized output by testing question answering over the resulting CSV dataset using a dataframe QA framework powered by locally hosted, lightweight LLMs that can be executed on a single GPU without reliance on proprietary cloud APIs. While QA is not the primary contribution, it provides an applied perspective on whether normalization enables reliable analytical queries with reduced manual effort.

1.3 Thesis Outline

The remainder of this thesis is organized as follows.

- **Chapter 2 (Problem Statement)** motivates the data integration problem in the permitting workflow, illustrates real-world spreadsheet heterogeneity, and presents the research questions addressed in this thesis.
- **Chapter 3 (Literature Review)** summarizes prior work on schema matching, spreadsheet understanding, LLM-based data preparation, scalability through retrieval and prefiltering, and downstream tasks such as question answering over structured datasets.
- **Chapter 4 (Data)** describes the collection of project-level comment-response spreadsheets, the key sources of heterogeneity across files, and the implications of wide- versus long-format representations for integration.
- **Chapter 5 (Methodology)** presents the end-to-end pipeline, including (i) sheet and header detection, (ii) description generation for schema elements, (iii) candidate prefiltering and Jellyfish-based schema matching, (iv) rule-based value-level normalization and wide-to-long transformation, and (v) question answering as a downstream task.
- **Chapter 6 (Results)** reports schema matching and normalization results under multiple experimental settings, provides error analysis, and discusses observed limitations and failure modes.
- **Chapter 7 (Conclusion and Future Work)** concludes the thesis, discusses broader implications for administrative data integration in regulatory contexts, and outlines future directions.

Chapter 2

Problem Statement

2.1 Context and Motivation

Environmental assessment and permitting processes generate large volumes of administrative data, often stored and exchanged as spreadsheets. In the BC mine permitting context, comment–response tracking spreadsheets record iterative review conversations between multiple actors, including government agencies (e.g., EMLI, ENV, First Nations), and project proponents. These records are essential for understanding review dynamics across projects and over time (e.g., how issues evolve across rounds, how long responses take, and which actors raise recurring concerns).

Despite their importance, these datasets are difficult to analyze systematically because they are not produced under a unified template. Instead, each project file encodes similar information using different structures, naming conventions, and layout decisions. As a result, even basic analyses require substantial manual preprocessing, and cross-project comparisons are often infeasible.

This thesis addresses this gap by formulating the data integration challenge as a schema matching and normalization problem and by developing a deterministic pipeline that transforms heterogeneous source spreadsheets into a unified long-format dataset suitable for analysis and querying.

2.2 Limits of Current Spreadsheet Schemas

The core limitation of current spreadsheet schemas is that they are designed for local project-level tracking rather than multi-project integration. Although most files contain conceptually similar fields (e.g., comment text, response text, dates, agency/author, review status), they lack consistent schema definitions.

Two sources of incompatibility are especially common:

Inconsistent column naming and semantics. Equivalent concepts appear under different column names (e.g., a field labeled “Document the Comment Pertains To” in one file corresponds to “Section” in another). In addition, the same concept may be encoded at different granularities across files. For example, one spreadsheet may store an identifier and agency name in separate columns, while another merges them into a single compound value (e.g., ENV-01) whose naming does not fully reflect its semantics. These inconsistencies prevent simple joins or merges across projects.

Inconsistent representation of review rounds. Multi-round conversations are represented either in wide format (multiple rounds stored as repeated column groups within a single row) or in long format (one row per round). Wide-format designs implicitly fix the maximum number of rounds and complicate integration across projects with different review structures, whereas long-format representations are more compatible with aggregation and longitudinal analysis.

2.3 Real-World Data Issues and Consequences

Beyond schema differences, real-world spreadsheet data introduces additional challenges that break straightforward integration:

- Multiple worksheets and non-data header/footer regions.
- Inconsistent agency naming and abbreviations that require canonicalization.
- Mixed date formats and partial or missing values that require normalization.
- Placeholder tokens (e.g., ‘‘x’’, ‘‘~’’, NaN, ‘‘-’’) that must be interpreted consistently.
- Multi-level headers (e.g., merged cells) that produce hierarchical column labels and must be flattened.
- Missing identifiers within multi-row conversations that must be propagated to maintain referential coherence.

These issues are not edge cases; they are structural characteristics of the data ecosystem. Without explicit preprocessing and normalization, analytical results may be biased toward the subset of projects that happen to

follow more consistent templates, and questions that require cross-project aggregation become prohibitively expensive to answer manually.

2.4 Problem Statement

The central problem addressed in this thesis is:

How can heterogeneous, project-specific permitting spreadsheets – with inconsistent schemas, layouts, and value encodings – be transformed into a unified, reproducible, and analyzable dataset that supports cross-project querying and downstream analysis?

This problem has two tightly coupled components:

1. **Schema alignment (column-level matching):** identifying which source columns correspond to which target schema fields despite naming and semantic variation.
2. **Value normalization (record-level transformation):** converting raw values into a consistent representation, including splitting compound values, normalizing dates, handling null tokens, and transforming wide-format round groups into long-format records.

2.5 Research Questions

This thesis is guided by the following research questions:

1. **Schema matching:** How can semantically equivalent columns be matched across heterogeneous spreadsheets in a way that is interpretable and reproducible?
2. **Deterministic generalization:** To what extent can a rule-based normalization pipeline generalize to previously unseen project files with new schemas and structural variants?
3. **Impact on analysis:** Does normalization meaningfully reduce the effort required to answer project- and process-level questions (e.g., counts by round, agency-level response time summaries, recurring topics)?
4. **Role of LLMs:** Where can language models help in schema matching without compromising determinism and auditability, and what are their practical limitations in this setting?

2.6 Motivation for This Work

This work is motivated by three key issues.

First, the underlying data problem is a practical bottleneck: valuable institutional knowledge is locked in inconsistent spreadsheets that resist scalable analysis. A unified dataset enables transparent, repeatable measurement of process characteristics (e.g., response delays, recurrence of issues, differences across actors and rounds).

Second, the problem requires careful engineering rather than purely statistical modeling. The challenge lies in making data reliable and comparable across projects, which is a prerequisite for any downstream analytics or decision support.

Third, the proposed approach is designed to be auditable. In administrative and regulatory settings, interpretability and traceability are essential: transformation rules must be explainable, outputs must be inspectable, and the pipeline must behave consistently across runs.

2.7 How Language Models Can Help Schema Matching

Language models can assist schema matching by providing semantic representations that are more informative than raw column names. In this thesis, language models are used as auxiliary components to enrich column descriptions (for both source and target schemas), enabling more reliable semantic comparison during matching.

However, language models are not used to directly “decide” normalization logic. The design goal is to preserve determinism and auditability: LLM outputs serve as structured context, while matching decisions remain explicit (e.g., binary decisions and post-processing rules for disambiguation).

2.8 Generalization of Rule-Based Normalization

A common concern with rule-based systems is whether they can generalize beyond known templates. This thesis treats generalization as a core requirement: the pipeline is developed using a subset of projects and evaluated on held-out project files. The emphasis is not on achieving perfect coverage of all possible schemas, but on providing a robust, extensible framework that

can incorporate additional rules as new structural patterns are encountered, while maintaining reproducibility.

2.9 Does Normalization Enable Question Answering?

A key motivation for normalization is to enable higher-level tasks that are otherwise difficult to perform on heterogeneous spreadsheets. Once data is represented under a unified long-format schema, analytical queries become significantly simpler and can be posed consistently across projects (e.g., “How many comments occurred in round 1 across all projects?”, “Which agency has the highest average response time?”).

This thesis therefore treats downstream querying and question answering as a validation of dataset usability: the normalization pipeline is not an end in itself, but a foundation that makes systematic analysis feasible.

2.10 Scope and Assumptions

This thesis focuses on deterministic schema matching and normalization of spreadsheet-based comment–response tracking data. It does not aim to build a fully automated end-to-end decision system. Evaluation emphasizes robustness, interpretability, and practical usefulness for analysis, rather than benchmark performance on generic schema matching datasets.

Chapter 3

Related Works

The field of data integration is undergoing a major transition from classical heuristic-based methods to more advanced neuro-symbolic and generative approaches. Schema matching is central to this evolution. This fundamental task involves identifying semantic correspondences between metadata elements across different data sources[2, 3]. Schema matching is essential for data warehousing, enterprise application integration, and creating interoperable machine learning-ready data[4]. Historically, schema matching has been recognized as a labor-intensive process and is often considered the most challenging and manual stage in modern database systems[3, 4]. The introduction of deep learning, especially transformer-based architectures, marked a significant shift toward semantic-aware automation. More recently, the reasoning capabilities of large language models (LLMs) have surpassed these earlier approaches. This chapter provides a comprehensive analysis of the literature on these developments, with a focus on the architectural shift from deep learning to LLMs, the contributions of key researchers, and the technical frameworks that enable local research and deployment.

3.1 Classical Schema Matching Approaches

The introduction of SMAT (Schema Matching via Transformers) in 2021 marked an important shift from traditional constraint-based and linguistic matching techniques. Before this, most schema-matching solutions relied on human-engineered rules, string similarity measures, or domain-specific dictionaries[3, 4]. While these methods worked well for simple one-to-one lexical mappings, they struggled with the semantic heterogeneity that is common in real-world enterprise databases.

3.1.1 SMAT Architectural Foundations and Contributions

SMAT proposes that schema matching—specifically the alignment of source schema elements to target schema elements—can be treated as inferring semantic relatedness or similarity between fields[3]. The model uses a deep neural network (DNN) with attention mechanisms[5], extending state-of-the-art natural language processing techniques to achieve semantic mappings using only attribute names and descriptions[3]. One of the key innovations of SMAT was its ability to capture semantic correlations without requiring explicit domain knowledge of the source and target systems, which significantly improved its deployment in different organizations[3].

The model’s performance was validated using the OMAP dataset, a benchmark based on real-world healthcare schema conversions[3]. SMAT treats the matching task as a classification or similarity scoring problem, using the context of data dictionaries to bridge the gap between different namings. SMAT’s influence can be seen in later research, which uses it as a baseline for more advanced models like SMOG (Schema Matching on Graphs), which uses iterative graph exploration to improve explainability in medical data integration[6].

Limitations of Deep Learning in Schema Matching

Although transformer-based models like SMAT improved schema matching, several structural limitations became clear as datasets grew in complexity. Deep learning (DL) approaches typically treat schema matching as a classification task, which requires a significant amount of labeled training data—a resource that is both expensive and difficult to obtain in specialized domains[7, 8]. Additionally, DL models like SMAT often perform better on textual or “dirty” data—where names are inconsistent—than on highly structured data, where they do not significantly outperform classical solutions[3].

Table 3.1 summarizes the performance and architectural trade-offs between traditional matching systems and the SMAT deep learning framework:

As researchers recognized these constraints, particularly the high cost of manual data labeling and the $O(n^2)$ complexity of comparing all possible attribute pairs, attention shifted toward retrieval-augmented and reasoning-based systems powered by large language models[9, 10].

3.2. Large Language Models for Schema Matching and Data Preparation

Feature	Traditional Matchers	SMAT (DL)
Input Metadata	Constraints, Types, Keys	Names, Descriptions
Learning Paradigm	Rule-based / Heuristic	Supervised Attention
Data Requirement	Low (Dictionaries)	High (Labeled Mappings)
Generalizability	Domain-specific	Cross-domain (Semantic)
Complexity Focus	Lexical Similarity	Contextual Relatedness
Explainability	High (Rule-based)	Low (Black-box DNN)

Table 3.1: Comparison of Traditional Schema Matchers and SMAT

3.2 Large Language Models for Schema Matching and Data Preparation

Large language models (LLMs) have changed how schema matching is done. LLMs can solve schema matching tasks because they contain vast knowledge from pre-training. They can identify errors, anomalies, and semantic matches in new datasets without human-engineered rules[9].

3.2.1 Philosophical and Technical Shifts Toward LLMs

LLMs are better than traditional DL models because they can work with zero-shot or few-shot reasoning. Unlike SMAT, which needs fine-tuning on labeled schema pairs, LLMs can start the schema matching process using only natural language instructions[7]. This is important in enterprises where privacy concerns prevent sharing source data for training.[10]

Models like ReMATCH (Retrieval Enhanced Schema Matching) exemplify this shift by treating the problem as a retrieval task rather than a classification task[10]. ReMATCH transforms schemas into structured documents and uses text embeddings to find candidate tables. This approach avoids the computational problems that come with traditional pairwise matching[10]. This design removes the need for predefined mappings or access to database rows, which improves both privacy and scalability[10].

3.2.2 Jellyfish: Specialized LLM Framework for Preprocessing

Jellyfish is an important model that solves multiple data preprocessing (DP) tasks: error detection, data imputation, entity matching, and schema matching[9]. Jellyfish has three configurations with different sizes, built

3.2. Large Language Models for Schema Matching and Data Preparation

Feature	Jellyfish (LLM)	SMAT (DL)
Base Model	Mistral-7B, Llama 3-8B, OOP2-13B	Transformer DNN
Primary Task	Instruction Following	Similarity Inference
Interpretability	High (NL Reasoning)	Low (Attention Weights)
Training Data	Instruction-tuning	Labeled Schema Pairs
Inference Mode	Retrieval & Reasoning	Pairwise Classification
Deployment	Local/Private GPU	Server-side

Table 3.2: Comparison of Jellyfish (LLM) and SMAT (DL) across key model components

on Mistral-7B[11], Llama 3-8B[12], and OOP2-13B[13] foundations. It is instruction-tuned using serialization and knowledge injection techniques designed for tabular data[9].

A unique characteristic of Jellyfish is its model interpreter, which provides natural language explanations for its decisions[9]. This reasoning capability allows it to outperform GPT-4 on seen datasets and maintain competitive performance on unseen datasets[9]. Jellyfish represents a broader trend: using local, single-GPU deployment to ensure data security while keeping the reasoning capabilities of larger commercial models[9].

The development from SMAT to Jellyfish shows a shift from simple similarity scores to "interpretive matching." In interpretive matching, the system explains why two attributes represent the same concept, which reduces the work data engineers must do to verify matches[9].

As shown in Table 3.2, Jellyfish shifts schema matching from supervised pairwise classification toward retrieval-driven reasoning with more interpretable, natural-language justifications.

Although Jellyfish demonstrates strong semantic reasoning capabilities, its practical deployment revealed certain limitations. In particular, matching quality may decline when column descriptions do not provide sufficient contextual information or when multiple candidate mappings are semantically similar. In such situations, additional preprocessing (e.g., richer description generation, embedding-based prefiltering) and post-processing rules were required to achieve consistent results. These observations indicate that while interpretive matching enhances transparency, reliable integration in heterogeneous real-world settings still depends on careful pipeline design beyond

the model itself.

3.3 Scaling Schema Matching: Scoping, Retrieval, and Reranking Pipelines

As the scope of schema matching expanded from simple one-to-one table mappings to complex multi-table enterprise schemas, researchers introduced multi-stage pipelines to manage the trade-off between runtime and accuracy.

3.3.1 Scoping, Configuration, and Human-in-the-Loop Perspectives

A recurring challenge in real-world schema matching is scalability in multi-source environments. When many heterogeneous schemas must be matched jointly, a large fraction of tables and attributes may be effectively *unlinkable* across sources, yet still occupy the search space and degrade both runtime and linkage quality. Recent work therefore emphasizes *pre-matching scoping* and *automatic tool configuration* as complementary mechanisms that reduce human effort while improving robustness.

Collaborative Scoping for Streamlined Schemas

Collaborative scoping introduces a self-supervised encoder–decoder framework for *linkability assessment* that prunes unlinkable schema portions *before* matching begins[14]. By enabling each local schema to scope its relevant tables and attributes independently, the method produces *streamlined schemas* that reduce the candidate space for downstream matchers. Experiments show that using streamlined schemas can substantially improve linkage quality in heterogeneous multi-source settings, supporting the broader view that scoping is a critical preprocessing phase rather than an optional optimization[14].

Automated Configuration of Schema Matching Tools

A second line of work addresses the difficulty of configuring schema matching systems that combine multiple matchers and aggregation strategies. Rather than relying on manual tuning, a reinforcement-learning (RL) approach has been proposed to learn tool configurations dynamically for a given matching setting[15]. This direction aligns with long-standing evidence that matcher

3.3. *Scaling Schema Matching: Scoping, Retrieval, and Reranking Pipelines*

selection and combination can be as influential as the underlying similarity signals, especially when schemas vary widely across domains[2].

Matchability Analysis and Human-in-the-Loop Refinement

Beyond algorithmic scalability, several studies emphasize that schema matching is inherently shaped by latent user intent and domain-specific business rules that may not be explicit in metadata. As a result, human-in-the-loop workflows remain important for understanding, validating, and refining mappings. Data-driven refinement paradigms use examples and interactive feedback to help users construct and revise complex transformations[16]. Complementarily, matchability analysis tools (e.g., mSeer) quantify how amenable a mediated schema is to being matched and provide actionable diagnostics for schema redesign[17].

3.3.2 The Magneto Hybrid Framework

Magneto addresses the computational cost of LLMs by combining them with Small Language Models (SLMs) in a two-phase retrieval and reranking pipeline[8]. In the retrieval phase, Magneto uses efficient SLM-based embeddings to identify potential candidate matches. In the subsequent reranking phase, a more powerful LLM (e.g., GPT-4o-mini or Llama 3) assesses these candidates using a broader semantic context[8].

To address the problem of lacking labeled training data for the SLM retriever, Magneto uses the LLM to generate syntactically diverse training data[8]. This self-supervised approach uses LLMs to create different representations for column names and values. These representations are then used to fine-tune the SLM through contrastive learning and triplet loss[8]. This approach allows Magneto to work with complex biomedical and proteomics datasets where traditional lexical matchers fail because of the ambiguity in attribute representations[8].

3.3.3 LLMatch and the SchemaNet Benchmark

Similarly, the LLMatch framework adds modularity to the schema matching process by dividing it into three parts: schema preparation, table-candidate selection, and column-level alignment[7, 18]. LLMatch is notable for its "Rollup" and "Drilldown" optimization strategies. The Rollup module consolidates semantically related columns into higher-order concepts, reducing the search space for the initial matching[7, 18]. The Drilldown module then

re-expands these concepts to perform fine-grained column mapping once a target table has been identified[18].

To evaluate these complex scenarios carefully, the creators of LLMatch developed SchemaNet, a benchmark with datasets from multiple domains (finance, healthcare, entertainment) created together with major financial institutions[18]. SchemaNet includes challenges like multi-table alignment and sparse metadata. Most existing benchmarks—which usually have only single tables—do not represent real-world enterprise integration needs[7, 18].

3.4 Question Answering on Structured Data

LLMs can query and reason over structured data in CSVs and dataframes. This research overlaps significantly with table understanding and data integration.

3.4.1 Benchmarks for Complex Analytical Reasoning

Text-to-SQL benchmarks often lack support for complex analytical reasoning because the query language is not expressive enough[19]. To address this gap, STARQA was created as a human-annotated dataset of complex analytical questions on specialized-domain databases[19]. TEXT2SQLCODE is another approach. It separates queries into two parts: SQL for data retrieval and Python for complex reasoning, such as calculations over aggregates[19].

NovelQA addresses long-context understanding by providing a benchmark where the context exceeds 200,000 tokens[20]. This is important for large CSVs where information is scattered across thousands of rows. Research shows that even advanced LLMs struggle to integrate information and recall evidence accurately in such long contexts[20].

3.4.2 Table Question Answering (TQA) Paradigms

TQA research now includes different table setups: provided tables or retrieved tables from large document collections[21]. A key challenge is processing hierarchical or multi-table structures. The model must understand relationships between tables and determine which tables can be joined[21].

A common approach is to build "custom agents" that use OpenAI functions to access Python REPLs (Read-Eval-Print-Loops) and retrievers for CSV queries[22]. These agents can choose between filtering rows in Python

or retrieving specific documents. This flexibility is essential for real-world analytical tasks[22].

3.5 LLMs and Spreadsheet Understanding

3.5.1 SpreadsheetLLM and SheetCompressor Framework

A key challenge in applying LLMs to spreadsheets is their 2D grid structure. Spreadsheets exceed token limitations and are not suitable for linear input formats[23]. SpreadsheetLLM addresses this by introducing SheetCompressor, a framework that reduces spreadsheet size while preserving structural information[23].

SheetCompressor includes three modules:

1. Structural-Anchor-Based Extraction: Identifies where tables begin and end, and which content regions are important[23].
2. Inverted-Index Translation: Converts the spreadsheet matrix format into a dictionary format, and merges cells with identical values to remove repetition[23].
3. Data-Format-Aware Aggregation: Groups adjacent cells that share the same format (such as currency or dates). This allows the LLM to understand patterns in the data without reading every single cell[23].

SpreadsheetLLM achieves strong results: a 25-fold compression ratio and 12.3% improvement in table detection compared to previous best models like TAPEX[24] and Binder[23, 25, 26].

3.6 Local LLM Research: Infrastructure and Model Specifics

For academic researchers and privacy-conscious organizations, the deployment of models on local infrastructure has become a viable alternative to commercial cloud APIs.

Local deployment is also relevant for document and spreadsheet extraction workflows that reconstruct structured data from semi-structured files. For example, TWIX reconstructs structured data from templated documents by inferring shared visual structure across records, illustrating an alternative to purely end-to-end vision-based LLM extraction[27].

3.6.1 The Role of the Ollama Framework

Ollama is a lightweight framework that facilitates the deployment of LLMs on local machines, offering enhanced privacy, reduced latency, and greater control over model behavior[28]. By eliminating the need for internet connectivity, Ollama ensures that sensitive data (e.g., healthcare or financial schemas) remains local[28].

Studies have shown that local deployment through Ollama significantly enhances learning outcomes and productivity[29]. Developers completed twice as many experimental iterations when using local models compared to pay-per-use APIs[29]. Furthermore, local deployment allows for the execution of agentic AI systems—which require many iterative queries—without the prohibitive costs of cloud tokens[29].

3.6.2 Mistral 7B: Efficiency and Architecture

Mistral 7B has emerged as a preferred model for local research due to its superior performance relative to its 7-billion-parameter size[11]. Its architecture introduces two key attention mechanisms that address the quadratic complexity of standard transformers:

1. Sliding Window Attention (SWA): Allows each token to attend to a maximum of W tokens from the previous layer, handled by a fixed-size rolling buffer cache[11].
2. Grouped-Query Attention (GQA): Accelerates inference speed and reduces memory requirements during decoding, allowing for higher batch sizes and throughput[11].

The architectural parameters of Mistral 7B enable it to outperform larger models like Llama 2-13B across benchmarks in common sense reasoning, world knowledge, and reading comprehension[11, 12]. Its coding and mathematical proficiency also approach those of much larger models, making it suitable for automating data analysis and code generation tasks[11].

Chapter 4

Data

The primary data used in this thesis consists of spreadsheet files (.xlsx) generated during environmental review and public consultation processes. Each spreadsheet contains comments submitted by various stakeholders, along with responses provided by regulatory bodies across one or more review rounds.

A key characteristic of the data is the lack of a standardized structure[cite: 427]. Common issues include:

- **Structural heterogeneity in review round representation**, ranging from multiple rounds embedded within a single row to rounds distributed across separate rows.
- **Inconsistent column naming conventions** across different project files.
- **Variation in the encoding of entities**, such as agencies, commenters, and identifiers.
- **Presence of non-data regions**, including titles, explanatory notes, and footers within the spreadsheets.

Although the semantic meaning of the data remains similar across files, these structural inconsistencies prevent direct aggregation or comparison. The raw data therefore cannot be used for analysis without substantial preprocessing.

4.1 Data Sources and General Characteristics

The data used in this thesis consists of a collection of Excel spreadsheet files (.xlsx), each corresponding to a single environmental assessment project. These spreadsheets document multiple rounds of written communication between government agency reviewers (e.g., EMLI, ENV, First Nations) and project proponents. Each round typically includes a comment submitted by

an agency and a corresponding response provided by the proponent, forming a conversational exchange over time.

Across projects, the spreadsheets contain largely similar types of information, including but not limited to: the author of the comment, the text of comments and responses, dates of submission and response, the project section to which the comment refers, and the subject or topic of the comment. Despite this semantic similarity, the datasets are highly heterogeneous in structure, which makes direct integration and analysis difficult. Across projects, the spreadsheets contain largely similar types of information, including but not limited to: the author of the comment, the text of comments and responses, dates of submission and response, the project section to which the comment refers, and the subject or topic of the comment. Despite this semantic similarity, the datasets are highly heterogeneous in structure, which makes direct integration and analysis difficult. Examples of these structural variations are provided in Appendix A.

4.2 Structural Heterogeneity and Integration Challenges

Although the spreadsheets describe conceptually similar data, they do not follow a common schema. Instead, they exhibit substantial variation in structure, column naming, and data representation. As a result, the datasets can be described as semantically consistent but structurally incompatible. This incompatibility is the primary motivation for the preprocessing and normalization steps developed in this thesis.

Two major sources of structural variation are observed across the files:

1. Inconsistencies in column naming and definition, and
2. Differences in how review rounds are represented.

4.2.1 Column Naming and Definition Inconsistencies

The first major source of heterogeneity arises from differences in column naming and column semantics. The same conceptual information is often represented using different column names across projects. For example, a column labeled “Document the Comment Pertains To” in one spreadsheet is equivalent to the column labeled “Section” in most other files. Similarly, columns such as “Rd 1 Open / Closed?” and “Round 1 Review Status”

encode the same information but use different naming conventions.

Beyond naming differences, some spreadsheets differ in how information is distributed across columns. In certain files, multiple columns are used to represent separate pieces of information, whereas other files merge the same information into a single column. For instance, one project includes two distinct columns; “ID #” and “Comment Organization”, which respectively represent a unique comment identifier and the reviewing agency. In contrast, another file contains a single column labeled “Comment ID3”, whose values take the form of compound strings such as “ENV-01”. These values encode both the agency name and the identifier within a single field, despite the column name suggesting only an identifier.

Furthermore, the identifier semantics differ across files: In some projects, identifiers are globally unique within the project, whereas in others they are only unique within each agency, resulting in repeated numeric identifiers across agencies (e.g., ENV-01, EMLI-01).

Such inconsistencies in value representation—where information is either fragmented across multiple attributes or consolidated into a single compound field—reflect a well-documented challenge in classical heterogeneous database integration [2]. For instance, similar to how an address may be represented as a single string or decomposed into distinct attributes, such as street, city, and postal code [2], the permitting data exhibits varying granularities of information density.

Historically, classical schema matching and data integration techniques have addressed these discrepancies using human-engineered rules, string similarity measures, or domain-specific dictionaries [4]. While effective for simple one-to-one lexical mappings, these traditional methods are limited by their reliance on explicit domain knowledge and struggle with the semantic heterogeneity found in real-world administrative datasets. Furthermore, these approaches typically require significant manual configuration and lack the flexibility to generalize across projects where naming conventions and structural layouts are not strictly enforced [2]. Consequently, these issues prevent straightforward aggregation and require the more robust, semantic-aware interpretation enabled by the LLM-assisted methodology presented in this thesis.

4.2.2 Representation of Review Rounds: Wide vs. Long Formats

The second major source of variation concerns how review rounds are represented in the spreadsheets. A single comment-response conversation may span one or more review rounds. Across the datasets, these rounds appear in two fundamentally different structural formats: **long** format and **wide** format.

In the long format, each review round of a conversation is represented as a separate row. Each row contains one comment-response pair, along with metadata identifying the review round. For example, a conversation involving three rounds would be represented by three rows, each corresponding to a single round. Table 4.1 illustrates an example of this structure.

ID	Section	Date	Agency	Phase	Comment	Response
EMLI.R1.02	Rec	25-Apr-23	EMLI	Round 1	[Comment R1]	[Response R1]
EMLI.R2.02	Rec	20-Jul-23	EMLI	Round 2	[Comment R2]	[Response R2]
EMLI.R3.02	Rec	22-Aug-23	EMLI	Round 3	[Comment R3]	[Response R3]

Table 4.1: Example of a long-format representation of a multi-round review conversation. Each row corresponds to a single comment-response pair within the same conversation.

In contrast, the wide format represents all review rounds of a conversation within a single row. In this structure, each round is stored in a repeated group of columns (e.g., Comment (Round 1), Response (Round 1), Comment (Round 2), Response (Round 2), and so on). As additional rounds occur, new column groups are appended to the spreadsheet. Due to the horizontal scale of these spreadsheets, Table 4.2 provides a truncated representation, using an ellipsis to indicate the continuation of round-specific columns that would otherwise exceed the page width.

Comment ID	Agency	Date	R1	Comment R1	Response R1	Date	R2	Comment R2	Response R2	...
EMLI-137	EMLI	31-Oct-23		[Comment R1]	[Response R1]	15-Jan-24		[Comment R2]	[Response R2]	...
EMLI-138	EMLI	10-Nov-23		[Comment R1]	[Response R1]	25-Jan-24		[Comment R2]	[Response R2]	...

Table 4.2: Example of a wide-format representation of a multi-round review conversation. All review rounds for a single conversation are stored in one row using round-specific columns.

While the wide format may appear convenient for incremental data entry, it

introduces significant challenges for integration and analysis. The number of review rounds is fixed implicitly by the number of columns, making it difficult to combine files with different maximum numbers of rounds. For example, integrating wide-format spreadsheets with three, four, and five rounds into a single static schema requires extensive restructuring. In contrast, long-format data maintains a consistent column structure regardless of the number of rounds, allowing review rounds to be treated as a dynamic attribute rather than a fixed structural component.

For these reasons, long-format is identified as the preferred target representation for integration and analysis.

4.3 Motivation for Normalization

The normalization process developed in this thesis transforms heterogeneous source spreadsheets that follow a tabular comment-response tracking structure, despite differences in schema, column naming, and round layout, into a unified long-format target schema. This process performs schema matching and structural normalization to integrate heterogeneous source files into a single consolidated dataset.

The transformation pipeline assumes that input spreadsheets follow a semi-structured tabular format in which comment-response information is represented in rows and columns, even if header organization or round layout varies. Specifically, the pipeline supports variations such as multiple worksheets, vertically stacked multi-row column headers (e.g., where a primary row designates "Round 1" and the subsequent row labels "Comment"), wide-format round groups, long-format round records, and compound identifiers embedded in cell values. However, spreadsheets containing row hierarchies with more than one level of nesting—where the semantic context of a record depends on a multi-layered parent-child row relationship—fall outside the intended scope of normalization. While the pipeline is designed to flatten multi-row column headers, it assumes a flat row structure for the data records themselves to maintain a consistent two-dimensional mapping. Also, non-tabular layouts or embedded free-form text regions fall outside the intended scope of normalization and would require additional preprocessing. These formats are particularly challenging to handle because irregular layouts and extensive free-form text often exceed the token limitations of many language models and fail to provide the structural anchors necessary for reliable automated parsing. Consequently, these multi-level row formats require

specialized extraction frameworks to isolate and flatten data regions before they can be integrated into the deterministic schema-matching pipeline.

This normalization enables two key outcomes. First, it significantly improves accessibility and analytical usability. Questions that are difficult or impractical to answer across multiple unstructured spreadsheets, such as computing the average number of comments per review round across all projects, become straightforward once the data is integrated into a single structured dataset.

Second, normalization serves as a critical preprocessing step for the broader analytical goals of this work. For example, to investigate potential sources of delay in the permitting process, it is necessary to analyze comment and response texts across projects, agencies, and review rounds. A unified and structured dataset provides a reliable foundation for such cross-project and longitudinal analysis, which would otherwise be infeasible given the diversity of the original spreadsheet formats.

4.4 Train and Test Data Separation

In addition to structural heterogeneity, the data used in this thesis presents challenges related to evaluation and generalization. Since the goal of the proposed pipeline is to normalize previously unseen spreadsheet files, it is essential to assess whether the designed preprocessing and schema matching logic generalizes beyond the specific files used during development.

For this reason, the dataset is explicitly divided into training and test subsets. This separation is not intended for predictive modeling, but rather for evaluating the robustness and transferability of the normalization pipeline when applied to new projects with unseen structures and schemas.

4.4.1 Definition of Training and Test Sets

The training dataset consists of a subset of project-specific Excel files used during the development of the pipeline. These files are employed to:

- Design the target schema
- Develop schema matching rules and configurations
- Define rule-based value transformations
- Identify common structural patterns and edge cases

4.4. Train and Test Data Separation

The test dataset consists of separate Excel files corresponding to different projects that are not used during pipeline development. These files may differ in column naming, structural layout, the number of review rounds, formatting conventions, These files may differ in column naming, structure, number of review rounds, formatting conventions, and inconsistent value representations (e.g., varying date formats or agency abbreviations). Importantly, no manual adjustments or schema-specific rules are introduced based on the test data. To ensure this isolation, a strict "frozen-pipeline" protocol was followed: all schema-matching configurations, rule-based value transformations, and LLM prompts were finalized and locked based exclusively on observations from the training subset. The test files were only processed through the end-to-end pipeline after development was complete, and their outputs were inspected post-hoc for validation purposes only. This methodology ensures that the evaluation reflects the pipeline's inherent robustness and its ability to generalize to novel spreadsheet structures without project-specific manual intervention.

4.4.2 Specification of Training and Test Files

Tables 4.3 and 4.4 present a transparent inventory of the experimental inputs, which supports the reproducibility of the study. Beyond simple enumeration, these tables highlight the **structural and longitudinal diversity** of the project-level data. By validating the pipeline on workbooks with different sheet configurations and record densities, the evaluation shows that the normalization logic generalizes across a representative cross-section of permitting records, rather than being over-fitted to a specific project template. This detailed specification at the worksheet level provides a robust baseline for future extensions of the framework. Because projects often include multiple sheets (e.g., a comment-tracking sheet alongside metadata sheets, or several comment-tracking sheets), the tables also indicate which sheet is used for analysis. The focus of this thesis is on one sheet per project for both the training and test subsets: the primary comment-tracking sheet when it is clearly designated, or otherwise the longest comment-tracking sheet among multiple candidates. This selected sheet is shown in bold. By specifying the dataset at the level of individual worksheets, the experimental inputs are clearly documented and the setup can be replicated or extended in future work.

The training subset is used during development to derive the target schema, define schema mappings, and implement rule-based transformations. The

4.4. Train and Test Data Separation

test subset is held out during development and is processed only after the pipeline is finalized, so it evaluates generalization on unseen spreadsheet structures. The complete collection of spreadsheets used in this research consists of non-public administrative permitting records provided by the Major Mines Office for research purposes. Due to their proprietary nature and the data-sharing conditions communicated at the time of access, the raw datasets are not publicly available.

#	Source file	Sheet	Data rows
1.	2021 01 25 NewAfton_B3_ITT.xlsx	Cover Page & Instructions REVIEW ITT References	59 147 38
2.	2021 02 10 NewAfton_B3_SSN_ITT.xlsx	Sheet1	69
3.	2021 06 13 BonanzaLedge_ITT.xlsx	Comment Tracking Lists	339 43
4.	2021 07 21 ElkGold_ITT.xlsx	Comment Tracking Lists	339 43
5.	2021 09 20 Brule_ITT.xlsx	Round 3 Final	278
6.	2021 09 23 QRMine_ITT.xlsx	Comment Tracking Lists	378 83
7.	2021 12 01 Premier_ITT.xlsx	Sheet1 Lists	660 43
8.	2022 08 16 NewAfton_CZone_ITT.xlsx	ITT - Technical	144
9.	2022 12 23 Blackwater_ITT.xlsx	Sheet3 Sheet2 AO Dropdown Comment Tracking Sheet	19 24 30 3863
10.	2023 08 31 GH0_Phase7-2_ITT.xlsx	EMLI ENV	56
11.	2023 11 LCO_ERX_CCR_Phase2_ITT.xlsx	10 LCO ERX CCR Mar 2 -Aug 10 2023	145
12.	2024 05 15 CGP_ITT.xlsx	CGP JPA App Review ITT	1739
13.	2025 01 03 Eskay BTS_ITT.xlsx	Export Summary Comment Tracking Application Updates - Tracking Generic Reponse Ex-amples	10 244 98 10

4.4. Train and Test Data Separation

#	Source file	Sheet	Data rows
		Comment Tracking Sheet	384

Table 4.3: Spreadsheet sources used for the training subset.

#	Source file	Sheet	Data rows
1.	2020_M-229_Thickener_ITT.xlsx	Cover Page & Instructions Pre-App ITT References Thickener Tech Review	59 85 38 50
2.	2020_M-243_WestZoneRamp_ITT.xlsx	Application Screening	23
3.	2021_M-11_HHDam_ITT.xlsx	Application Review Application Screening Status choices Sheet3	29 19 6 1
4.	2021_M-173_TSF Buttrass_ITT.xlsx	Tracking Sheet Sheet3	22 1
5.	2021_M-229_ElkGold_ITT.xlsx	NA GM CM ITT	7
6.	2021_M-240_UGBoxcut_ITT.xlsx	IRT Screening Application Review Status choices Sheet3	22 42 32 6 1
7.	2021_M-40_GibEast_ITT.xlsx	owssvr	88
8.	2022_M-147_TSFButtrass_ITT.xlsx	Tracking Sheet Status choices Sheet3	25 4 1
9.	2022_M-147_TSFraise_ITT.xlsx	Tracking Sheet Status choices Sheet3	15 4 1
10.	2022_M-238_CowMBBR_ITT (2).xlsx	Comment Tracking Lists	145 43
11.	2022_M-238_CowMBBR_ITT.xlsx	Comment Tracking Lists	145 43

4.4. Train and Test Data Separation

#	Source file	Sheet	Data rows
		Sheet3	1
12.	2022_M-242_ExplorationPhase1_ITT.xlsx	Comment Tracking Lists	89 45
13.	2022_M-242_NPAGRock_ITT.xlsx	Comment Tracking Lists	112 45
14.	2022_M-40_PreProduction_ITT.xlsx	Permitting Path PD&IRT Screening App Review 5Yr Costing 2021 ARR Terms	14 33 19 144 61 24 8
15.	2023_C-221_SRFStudy_ITT.xlsx	ITT IR Concordance	59 7
16.	2023_C-223_HermannMajor_ITT.xlsx	owssvr (32)	1057
17.	2023_M-112_RCP_ITT.xlsx	Ajax (M-112) RCP First Nations	37 8
18.	2023_M-179_PNLPortal_ITT.xlsx	PNL MAPA Disclaimer Comment Tracking Sheet	44 1 49
19.	2023_M-197_ Warehouse_ITT.xlsx	Disclaimer Comment Tracking Sheet	1 49
20.	2023_M-200_SERDS_ ITT.xlsx	Comment Tracking Sheet	28
21.	2023_M-229_BullRiver_ITT.xlsx	CM Bull River ITT	10
22.	2023_M-245_PAGPMA_ITT.xlsx	Disclaimer Comment Tracking Sheet Email comment track- ing	1 32 3
23.	2023_M-40_ StarterDam_ITT.xlsx	Review	11
24.	2023_M-40_AWTP_ITT.xlsx	Review Screening	125 69
25.	2023_M-68_OreStockpile_ITT.xlsx	Tracking Sheet Status choices Question 4 Info Re- quest	10 4 9

4.4. Train and Test Data Separation

#	Source file	Sheet	Data rows
26.	2024_C-156_LittleWindy_ITT.xlsx	Comment Tracking Sheet	168
27.	2024_C-223_CoalImport_ITT.xlsx	Disclaimer Comment Tracking Sheet	1 36
28.	2024_M-11_MLE_ITT.xlsx	Explanatory Comment & Response Sheet1	21 4587
29.	2024_M-179_DelayAmd_ITT.xlsx	Sheet1	59
30.	202_M-200_TSF973_ITT.xlsx	EMLI Geotech-974 Raise EMLI Reclamation-974 Raise EMLI Geoscience-974 Raise XFN Comments - 974 Raise	15 33 22 8
31.	2025_M-236_Buttreass_ITT.xlsx	Disclaimer Comment Tracking Sheet	1 30
32.	2025_M-237_DomeRestart_ITT.xlsx	Sheet1	200
33.	2025_M-242_ExplorPhase2_ITT.xlsx	Comment Tracking Sheet Lists	444 47
34.	2025_M-243_ProductionRate_ITT.xlsx	Review Comment Tracking Lists	85 11
35.	2025_M-40_SaddleDam_ITT.xlsx	Disclaimer Comment Tracking Sheet	1 20

Table 4.4: Spreadsheet sources used for the test subset.

4.4.3 Rationale for Project-Level Splitting

The train-test split is performed at the project level, rather than at the row or record level. This design choice reflects the real-world deployment

scenario of the pipeline, where an entire spreadsheet from a new project must be processed without prior knowledge of its structure.

Splitting data at the row level would result in information leakage, as rows from the same spreadsheet typically share identical schemas and structural conventions. Such a split would therefore provide an overly optimistic evaluation of the pipeline’s performance.

By contrast, project-level splitting provides a more realistic assessment of generalization. Manual inspection of the dataset confirmed that the test projects introduced novel column naming conventions, unique agency abbreviations, and distinct structural variants—such as specific wide-format round encodings—that were not present in the training subset. This ensures that the evaluation measures the pipeline’s ability to interpret and normalize previously unseen metadata and value representations, rather than relying on a training set that spans the entire space of possible formats.

4.4.4 Role of the Train-Test Split in the Pipeline

The train-test separation is used exclusively for evaluation and validation purposes and does not correspond to the training of a predictive model. Instead, the split separates the data used for pipeline development from the data used for final evaluation.

During development, the normalization pipeline is iteratively refined using only the training projects. Once the pipeline design is finalized, it is applied to the test projects without modification in order to evaluate its ability to generalize to previously unseen spreadsheet schemas.

In this context, the term “training” refers to pipeline development and rule refinement rather than model parameter learning.

The outputs generated from the test set are inspected to assess:

- Whether source columns are correctly mapped to the target schema
- Whether value-level transformations behave consistently
- Whether wide-format and long-format inputs are normalized correctly
- Whether previously unseen column names and structures are handled correctly

This evaluation approach focuses on the correctness and robustness of the

normalization process across heterogeneous schemas, rather than on predictive performance metrics typically used for trained models.

4.4.5 Limitations

It is important to note that the absence of a formal ground-truth mapping limits the evaluation to qualitative and manual inspection. Since not all source columns are expected to map to the target schema, and some columns may be project-specific, automated accuracy metrics are not applicable.

Nevertheless, the train-test split provides a structured framework for assessing whether the pipeline can generalize to new projects without requiring manual intervention.

Chapter 5

Methodology

The methodology proposed in this thesis is centered around a modular data normalization pipeline designed to handle highly heterogeneous Excel-based datasets. The primary objective of the pipeline is not full automation, but rather **robustness, transparency, and generalizability** when processing previously unseen project files. By adopting a human-in-the-loop (HITL) approach, the pipeline allows for human intervention at critical junctures: (i) the manual definition and refinement of the target schema and its semantic descriptions, (ii) the configuration of deterministic, rule-based transformation logic for domain-specific value normalization, and (iii) the post-hoc validation of the mapping selection stage to resolve complex ambiguities that semantic matching alone cannot address. This design ensures that while the LLM-assisted components handle high-dimensional semantic inference, the human expert maintains ultimate control over the transformation logic and final data integrity.

The pipeline is implemented in Python and structured as a sequence of clearly defined stages. Each stage performs a specific transformation and produces intermediate outputs that can be independently inspected and validated. This modular design reflects the real-world constraints of the data, where assumptions about schema consistency cannot be made and silent failures may lead to incorrect analytical conclusions.

Figure 5.1 summarizes the end-to-end normalization pipeline used in this thesis. Starting from raw, heterogeneous Excel files, the pipeline performs structural cleaning, generates semantic descriptions for both source columns and target schema fields, and then combines an embedding-based prefiltering step with Jellyfish’s binary schema matching to obtain candidate mappings. Post-processing resolves ambiguities among closely related fields and selects a final mapping per source column, after which deterministic value-level transformations and a wide-to-long conversion produce a unified long-format dataset. The final block illustrates a downstream question-answering

5.1. Input Identification and Structural Cleaning

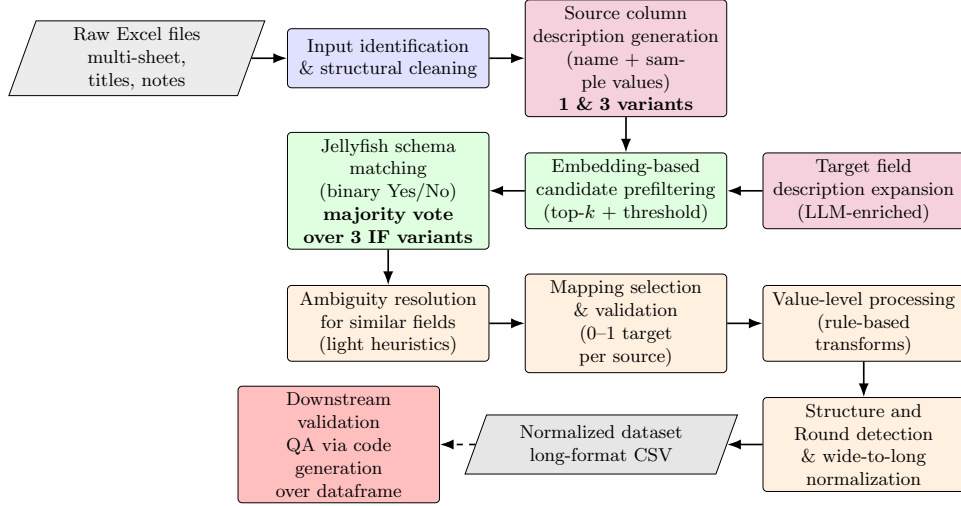


Figure 5.1: Flowchart of the modular normalization pipeline, from raw Excel inputs to a unified long-format dataset. The dashed arrow indicates the optional downstream QA validation stage.

task used to validate that the normalized representation supports reliable analytical queries.

5.1 Input Identification and Structural Cleaning

The first stage of the pipeline focuses on preprocessing raw Excel files. These files often contain multiple worksheets, as well as non-tabular elements such as titles, explanatory notes, or summary rows. As a result, the location of the actual data table cannot be assumed to be fixed across files.

5.1.1 Sheet and Header Detection

To robustly identify the correct table region, the pipeline applies a conservative multi-step procedure. When an Excel file is first loaded, the system reads the first worksheet by default and initially assumes that it contains the main data table with the header row at the top. This assumption is treated as a starting point rather than a guarantee: subsequent checks are used to confirm that the selected sheet and header row form a valid tabular structure.

This stage prioritizes correctness over aggressive automation. Concretely,

the pipeline applies the following conservative heuristics:

- **Sheet Selection:** The system identifies the primary comment-tracking sheet by evaluating row density across all worksheets; if multiple candidates exist, it selects the sheet with the highest count of non-empty rows.
- **Header Identification:** To distinguish the header from metadata or titles, the pipeline scans the initial rows and selects the first row that contains a minimum threshold of unique, non-null strings, ensuring it aligns with the expected width of a permit tracking table.
- **Boundary Validation:** The detected table region is validated by checking for a consistent number of columns across subsequent rows; if the row structure becomes highly irregular or empty, the pipeline treats these as non-data regions and truncates the parsing range accordingly.

Conservative heuristics are used to avoid misidentifying headers or data boundaries, since errors introduced at this step propagate to all subsequent stages.

5.1.2 Jellyfish Input Preparation

After a valid sheet and header set are detected, the pipeline prepares structured inputs for the downstream schema matching stage. For each column in the selected sheet (as identified by its header), the system extracts: (i) the source column name, and (ii) representative sample values to provide semantic context.

The pipeline then retrieves the predefined target schema, consisting of standardized field names and their expected semantic descriptions. Given a source column, candidate mapping pairs are constructed by combining its extracted representation with each target field description. Each candidate pair is formatted into a consistent record containing the source identifier, sampled context, the target identifier and description, and a source-file reference to preserve provenance. These records are written to a CSV file that is directly consumed by Jellyfish for binary semantic equivalence evaluation.

Finally, the pipeline removes duplicate candidate pairs within a file (i.e., repeated source–target combinations) to reduce redundancy and unnecessary downstream computation. Importantly, identical source–target pairs are *not* removed across different files, since sample values may differ by project and can influence the semantic matching outcome.

5.2 Column-Level Schema Matching

5.2.1 Motivation and Problem Definition

A core challenge addressed in this thesis is the lack of structural consistency across project-specific Excel files. Although different spreadsheets contain semantically similar information, they use heterogeneous column names, definitions, and layouts. As a result, it is not possible to directly align source columns across projects using string matching or fixed positional assumptions. Manual inspection of the datasets revealed that equivalent concepts are frequently labeled with semantically distinct headers (e.g., “Document the Comment Pertains To” vs. “Section”) or encoded at different granularities, such as compound identifiers (ENV-01) that merge agency and ID into a single string. Furthermore, the transition between wide and long formats across projects—where review rounds are either stored as repeated column groups or independent rows—creates a structural misalignment that prevents simple joins or fixed-index mapping. These fundamental inconsistencies necessitate the semantic-aware interpretation provided by the LLM-assisted methodology used in this thesis.

To address this problem, schema matching is performed at the *column level*, where each source column is evaluated for semantic equivalence with a pre-defined target schema field. This stage is responsible for determining what each column represents, independent of how its values are later processed.

5.2.2 Target Schema Design

The target schema represents the desired unified structure of the normalized dataset. It is defined independently of any single project and reflects the conceptual entities required for downstream analysis, such as comment metadata, review round information, and organizational context.

Each target field is associated with a brief textual description that specifies its intended semantic meaning. These descriptions serve two purposes: they document the schema for human understanding and act as inputs to the schema matching process. Table 5.1 summarizes the target schema fields used in this thesis and their semantic meanings.

5.2.3 Column Description Generation

Before schema matching can be performed, both source and target columns must be represented in a form suitable for semantic comparison. Since

5.2. Column-Level Schema Matching

Target field	Semantic meaning
<code>comment_id</code>	Unique identifier for the comment
<code>section</code>	Document section being referenced
<code>topic</code>	Topic or theme of the comment
<code>round</code>	Review round number
<code>comment_type</code>	Type of comment (e.g., Information Requirement)
<code>comment_text</code>	Full text of the comment
<code>response_text</code>	Proponent’s response to the comment
<code>author</code>	Author of the comment
<code>date_received</code>	Date comment was received
<code>date_responded</code>	Date response was sent
<code>status</code>	Current status of the comment
<code>agency</code>	Agency (e.g., EMLI, First Nations)
<code>project</code>	Project name

Table 5.1: Target schema used for normalization, with field-level semantic definitions.

raw column names alone are often ambiguous, abbreviated, or project-specific, additional contextual information is required to capture their intended meaning.

For this reason, a **column description generation step** is introduced for both source and target schemas. These descriptions serve as semantic representations that enable meaningful comparison during schema matching.

Source Column Description Generation

For each source column, a textual description is generated by combining:

- The column name
- Available contextual cues from sample values

The resulting description aims to capture the semantic intent of the column rather than its exact formatting or naming. This step is critical for reducing ambiguity introduced by inconsistent naming conventions and project-specific abbreviations.

In practice, two additional sources of variability must be handled. First, the same column name may appear with different sample values across project

files, which can change the inferred semantics of the column. Descriptions are therefore generated per unique (*column name, sample-value set*) combination rather than per column name alone.

Second, even with a low sampling temperature, LLM outputs can exhibit minor nondeterministic variation (e.g., wording choices) that can affect downstream semantic matching. To improve robustness, the pipeline generates **three** independent description variants for each source column instance and stores them as `source_description_1`, `source_description_2`, and `source_description_3`. These variants are later used jointly during schema matching to reduce sensitivity to any single generated phrasing.

Target Schema Description Generation

Although each target schema field is initially associated with a brief textual description, these descriptions are intentionally concise and serve primarily as high-level documentation. In practice, such short descriptions are insufficient for reliable semantic matching against heterogeneous source columns.

To address this limitation, extended descriptions are generated for target schema fields. These enriched descriptions provide additional contextual information and clarify the intended semantic scope of each field. For example, a target field labeled **section** with a brief high-level description such as “*Document section being referenced*” is expanded into a more descriptive representation: “*This column identifies the specific document section being discussed in the comments exchanged between reviewers and proponents during the major mine project review process.*” The goal of this step is not to redefine the target schema, but to supplement it with sufficient context to support accurate schema matching.

Language Model Selection

An LLM is used in this work to generate short natural-language descriptions for spreadsheet columns, based on the column name and a small set of representative sample values. Because these descriptions are later consumed by the downstream schema matching component, the main requirement is not creative text generation, but *stable, semantically focused, and cleanly formatted* outputs that avoid speculative assumptions.

To select a suitable local model, four candidate LLMs were evaluated using a small benchmark extracted from real project spreadsheets: (i) `llama3.1:8b`, (ii) `mistral:7b`, (iii) `neural-chat`, and (iv) `qwen3:8b`. The evaluation set

includes 10 representative source columns, spanning different data types (free text, categorical labels, and dates). For each column, the prompt provided the column name together with three sample values. The generated descriptions and runtimes are archived to facilitate qualitative comparison.

Model selection was based primarily on qualitative criteria: (1) semantic correctness (does the text capture the intended meaning of the column?), (2) clarity and conciseness (is the description readable and focused?), (3) robustness and formatting discipline (does it avoid emitting extraneous meta-text such as internal reasoning?), and (4) suitability for repeated local execution. Table 5.2 shows a single representative example for the source column **Date Received**, for which the four models exhibit noticeably different output quality. In particular, this example highlights differences in formatting discipline (e.g., emitting internal reasoning) and in the tendency to introduce unsupported assumptions.

Across the 10-column benchmark, **qwen3:8b** frequently produced outputs containing explicit chain-of-thought markers (e.g., `<think>`) and, in multiple cases, did not reliably return a clean final description. This behavior introduces avoidable noise into the pipeline and would require additional post-processing to sanitize outputs before schema matching. **neural-chat** generated consistently well-formed English, but its descriptions were often formulaic (e.g., repeated “This database column...” templates) and sometimes added generic statements with limited semantic benefit. **llama3.1:8b** produced concise descriptions, but it tended to hedge heavily (e.g., “appears to”) and occasionally introduced stronger assumptions than warranted by the evidence in the samples (for example, inferring that a field is “required” because all observed samples were **Yes**).

In contrast, **mistral:7b** most consistently generated descriptions that were (i) semantically informative, (ii) cleanly formatted for direct downstream use, and (iii) focused on the column’s meaning without relying on speculative claims. Although **mistral:7b** outputs were slightly longer on average than the other candidates in this benchmark, the additional content generally improved interpretability (e.g., by explicitly stating the categorical nature of a field and its role in organizing records). For these reasons, **mistral:7b** was selected as the default model for description generation in the normalization pipeline.

5.2. Column-Level Schema Matching

LLM	Time (s)	Generated description (full text)
llama3.1:8b	50.3	The "Date Received" column represents a timestamp that indicates when each record was received, containing date values with no apparent time zone information. From the sample values, it appears to be tracking dates in December 2020 and January 2021, suggesting it might be used for logging or recording events as they are received over time.
mistral	50.4	This "Date Received" column contains timestamp data representing the exact date and time when an event, possibly related to transactions or records, occurred within the system. In the provided sample data, it appears that the events were recorded on December 14, 2020, January 5, 2021, and December 17, 2020, suggesting a pattern of irregular but recurring entries.
neural-chat	42	This "Date Received" column contains time stamps representing when certain events or items were received, typically indicating their arrival date. It serves to track and organize occurrences chronologically within a database. The sample values showcase varying months and years, with no apparent pattern, suggesting diverse receipt dates.
qwen3:8b	59.5	<think> Okay, the user wants me to describe the "Date Received" column based on the sample data provided. Let me start by looking at the samples: 2020-12-14, 2021-01-05, 2020-12-17. All of these are dates, so the data type is definitely dates or timestamps. The column name suggests it's about when something was received.

Table 5.2: Example outputs for one source column (**Date Received**) generated by four candidate LLMs using identical prompts (column name + sample values), together with per-call generation time.

Prompt Design and Evaluation

The quality of generated descriptions is highly dependent on prompt design. Multiple prompt templates were evaluated to assess their impact on description clarity, specificity, and consistency across different columns.

The prompts were designed to encourage the model to focus on semantic intent rather than surface-level formatting. Differences in prompt structure led to noticeable variations in output quality, including levels of detail, inclusion of assumptions, and consistency across similar columns.

The final prompt template was selected based on qualitative evaluation of the generated descriptions. The evaluation focused on whether the out-

puts provided sufficient contextual information to support accurate schema matching while remaining concise and interpretable.

This prompt-based evaluation highlights the importance of controlled prompt design when using language models as auxiliary components within a deterministic preprocessing pipeline.

Full prompt templates are provided in Appendix B (Section B.1).

5.2.4 Jellyfish Schema Matching

Schema matching is performed using an external semantic matching tool, referred to as *Jellyfish*. Jellyfish evaluates pairs of column descriptions and determines whether they are semantically equivalent. In this pipeline, Jellyfish is used exclusively at the schema level and operates on the generated textual descriptions of source and target columns.

Rather than querying Jellyfish against the entire target schema of 12 fields for every source column, the pipeline first applies an embedding-based pre-filtering step to reduce the number of expensive model inferences. Jellyfish is then run only on a small candidate set (top $k = 5$) of target fields per source column. This reduction is critical for the pipeline’s performance, particularly given the use of three description variants per column. On average, each Jellyfish mapping takes 1.90 seconds; by limiting the candidate set, the inference time is reduced from 22.8 seconds to 9.5 seconds per source column, representing a saving of over 58% in processing time per column.

Instead of returning a similarity score or ranked list, Jellyfish produces a **binary decision** (Yes/No) indicating whether the source column matches a given target field.

Because three source description variants are available, each candidate (source, target) comparison is evaluated three times—once per source description variant. The final Jellyfish decision is obtained by majority vote over the three binary outputs.

This design choice is intentional. Binary outputs avoid threshold tuning and reduce ambiguity in downstream processing. They also align with the goal of maintaining explicit and interpretable mapping decisions.

The output of the Jellyfish matching stage is a set of candidate mappings between source columns and target schema fields.

Embedding-Based Candidate Prefiltering

Jellyfish inference is comparatively slow when applied to the full Cartesian product of source and target fields. To make schema matching scalable across many project files, the pipeline performs a lightweight semantic retrieval step before invoking Jellyfish.

Each source and target field is represented as a short text formed by concatenating the column name with its generated description. Sentence embeddings are computed using a compact transformer embedding model, and cosine similarity is used to measure semantic proximity between each source column and all target fields. For each source column, only the top- k most similar target fields are retained, subject to a minimum similarity threshold.

Because three source description variants are available, embeddings are computed for each variant and the *maximum* similarity across the three representations is used during filtering. This design reduces the risk of filtering out a correct match due to an unlucky description phrasing, while still substantially decreasing the number of Jellyfish queries.

Model Size Selection

Jellyfish is available in multiple parameter configurations, including models with approximately 7 billion, 8 billion, and 13 billion parameters. Selecting an appropriate model size involves balancing semantic accuracy, computational cost, and output stability.

In this work, the 7-billion-parameter variant is selected. Empirical comparison between the 7B and 8B models showed that the larger model did not consistently improve matching accuracy for the schema matching task. In several cases, the 8B model produced more verbose or less stable outputs without providing clearer semantic distinctions. Given the relatively constrained and repetitive nature of column descriptions, the additional capacity of larger models did not translate into meaningful performance gains.

The 7B model was therefore chosen as it provided comparable or better matching consistency while remaining computationally efficient and suitable for repeated local execution.

Prompt Design for Schema Matching

Multiple prompt formulations were evaluated for the Jellyfish matching task. These prompts varied in how they presented column descriptions, requested

reasoning, and constrained the final output.

Effect of Chain-of-Thought Prompting Prompting strategy was found to have a substantial impact on Jellyfish’s matching performance. In particular, incorporating chain-of-thought reasoning into the prompts led to a noticeable improvement in semantic discrimination between closely related columns. By encouraging the model to reason explicitly about the meaning of each column before producing a final Yes/No decision, chain-of-thought prompting reduced ambiguous matches and improved consistency across similar schemas. While quantitative evaluation is left for future work, qualitative inspection showed clear improvements in alignment accuracy when chain-of-thought prompting was enabled.

The final prompt template was selected based on its ability to encourage careful semantic comparison while maintaining a concise and deterministic Yes/No response format. Full prompt templates are provided in Appendix B (Section B.2).

Probability-Based Matching Attempts In addition to binary schema matching, alternative prompt formulations were explored in which Jellyfish was asked to output, alongside the Yes/No decision, a confidence score in the range $[0, 1]$ indicating the model’s estimated probability of a correct match. The motivation for this approach was to use confidence values to resolve cases where a single source column was mapped to multiple target fields.

In practice, this approach proved ineffective. Across a wide range of test cases, the reported confidence values were consistently high, typically 0.95, even in situations where a source column was mapped to multiple semantically similar target fields. This behavior was observed both when the final binary decision was correct and when the mapping was ambiguous or incorrect.

As a result, the probability scores did not provide meaningful additional information beyond the binary decision. The lack of variability in confidence values made them unsuitable for ranking or disambiguation purposes.

Consequently, probability-based outputs were removed from the final prompt design.

5.2.5 Handling Ambiguity Between Similar Fields

A known limitation of semantic schema matching arises when multiple target fields are highly similar in meaning, such as distinguishing between *comment text* and *response text*. In such cases, Jellyfish may correctly identify semantic similarity but fail to reliably distinguish between closely related fields.

To mitigate this limitation, an additional post-processing step is applied after schema matching. When multiple target fields are identified as positive matches for a single source column, string-based heuristics are used to disambiguate the mappings.

Concretely, the pipeline implements a deterministic selection strategy:

- **Name Normalization:** Both source and target column names are normalized by converting to lowercase and removing special characters to ensure robust comparison (e.g., `Comment.Text` and `comment-text` both become `comment text`).
- **Similarity Scoring:** The pipeline calculates a similarity ratio between the normalized source name and each candidate target name using the *SequenceMatcher* algorithm. For example, if a source column `comment_msg` is mapped by Jellyfish to both `comment_text` and `response_text`, the pipeline selects `comment_text` due to its higher lexical similarity score.
- **Composite Key Grouping:** To ensure project-specific accuracy, disambiguation is performed within groups defined by a composite key of the source column name, the source file, and specific sample values. This prevents valid mappings in one project from being incorrectly applied to similar-looking columns in another.
- **Target-Specific Deduplication:** For critical metadata fields that should appear only once per project (e.g., `comment_id`, `agency`, `author`), a final deduplication rule ensures only the first valid mapping is retained.

This hybrid approach combines semantic matching with lightweight string matching to resolve ambiguities that semantic models alone cannot reliably address, without altering the core schema matching logic.

5.2.6 Mapping Selection and Validation

Since not all source columns are expected to map to the target schema, and some columns may be project-specific or irrelevant, the pipeline does not assume a one-to-one correspondence.

The mapping selection process follows explicit rules:

- A source column may map to at most one target field.
- Source columns with no positive match from Jellyfish are excluded from the normalization process to ensure high precision.
- In cases where Jellyfish identifies multiple positive matches for a single source column, the pipeline applies the automated string-similarity priority rules described in the previous subsection. Manual inspection of the training results indicated that such ambiguities arose periodically, most frequently when distinguishing between `comment_text` and `response_text` or between various date fields (e.g., `date_received` vs `date_responded`).

This approach reflects the absence of a complete ground truth and acknowledges that schema matching in this context is inherently partial. The pipeline therefore emphasizes correctness over coverage.

5.2.7 Design Rationale and Limitations

The combination of description generation and binary schema matching reflects the practical constraints of the data. Given the lack of standardized schemas, inconsistent documentation, and absence of labeled ground truth, probabilistic or fully automated approaches would introduce uncertainty that is difficult to validate.

By contrast, the approach adopted in this thesis provides a more controlled and interpretable mechanism for aligning heterogeneous schemas while remaining extensible. Future work could explore alternative matching strategies or confidence-based ranking, but such extensions are considered outside the scope of this thesis.

5.3 Value-Level Processing and Rule-Based Transformations

After column-level matching, the pipeline processes individual cell values to resolve inconsistencies that cannot be addressed at the schema level. This stage is implemented entirely using explicit, rule-based logic.

5.3.1 Normalization Procedure

The value-level normalization procedure operates on the mapped source columns and transforms raw spreadsheet values into a standardized representation aligned with the target schema. The procedure consists of the following steps:

- Standardization of agency names using consistent abbreviations.
- Decomposition of compound values (e.g., agency and identifier encoded in a single field) into atomic attributes.
- Normalization of date formats into a consistent canonical representation.
- Canonical handling of null or placeholder values (e.g., ‘`x`’, ‘`~`’, NaN, ‘`-`’).
- Propagation of missing `comment_id` values across multi-row conversations.
- Flattening of multi-level column headers produced by merged cells or hierarchical spreadsheet layouts.
- Extraction of project identifiers from source filenames to ensure traceability.

Each transformation step is applied sequentially and independently, allowing intermediate outputs to be inspected and validated. This design choice prioritizes transparency and correctness over maximal automation.

5.3.2 Handling of Review Rounds and Wide-to-Long Normalization

A critical component of value-level processing is the automatic detection of review rounds and the transformation of data from wide to long format.

5.3. Value-Level Processing and Rule-Based Transformations

Source spreadsheets often encode multiple rounds of a comment–response conversation within a single row, using repeated column groups.

For each detected conversation, the pipeline identifies shared metadata (e.g., agency, section, topic) and replicates these attributes across all extracted rounds. Round-specific attributes, such as comment text, response text, dates, and review status, are extracted into separate records. As a result, each review round is represented as an independent record in the normalized dataset.

This transformation enables consistent representation of conversations with varying numbers of rounds and ensures compatibility across projects with different structural conventions.

5.3.3 Target Schema and Output Representation

The resulting unified dataset represents each comment-response pair as a single record in a standardized long-format schema. Core fields include `comment_id`, `agency`, `round`, `comment_text`, `response_text`, and `source_file`.

The final output of the normalization process is written to CSV files. CSV is selected to facilitate compatibility with downstream analytical workflows and statistical tools, while preserving a simple and inspectable representation of the normalized data.

5.3.4 Example: Wide-to-Long Normalization

To illustrate the effect of value-level normalization and wide-to-long transformation, Tables 4.2 and 5.3 present a concrete example of a multi-round review conversation before and after normalization.

comment_id	agency	round	date_received	comment	response	status
EMLI-137	EMLI	1	2023-10-31	[Comment R1]	[Response R1]	Open
EMLI-137	EMLI	2	2024-01-15	[Comment R2]	[Response R2]	Open
EMLI-137	EMLI	3	2024-02-28	[Comment R3]	[Response R3]	Closed
EMLI-137	EMLI	4	2024-04-04	[Comment R4]	[Response R4]	Closed

Table 5.3: Example of the normalized target schema. Each record represents a single comment–response pair, regardless of the original source format.

Table 4.2 shows a wide-format representation of a review conversation, in which all review rounds associated with a single comment identifier are

stored within a single row. Each review round is represented using a separate group of round-specific columns (e.g., comment text, dates, and status). While this representation may be convenient for manual data entry, it implicitly fixes the maximum number of rounds and complicates integration across projects with different review structures.

In contrast, Table 5.3 shows the corresponding normalized long-format representation produced by the pipeline. Each review round is extracted into a separate record, while shared metadata such as `comment_id` and `agency` are replicated across rounds. As a result, a four-round conversation originally stored in a single wide-format row is transformed into four independent records, one per review round.

This transformation enables consistent representation of review conversations with varying numbers of rounds and facilitates downstream analysis across projects, agencies, and review stages. Without this normalization step, systematic comparison of comment evolution and response dynamics would not be feasible.

5.3.5 Rationale and Limitations

Rule-based value-level processing is intentionally chosen due to the availability of domain knowledge and the absence of reliable ground-truth mappings. Deterministic logic ensures reproducibility and makes transformation decisions auditable.

While this approach requires manual specification of transformation rules, it avoids the uncertainty introduced by probabilistic methods and ensures that normalization behavior remains stable across different runs and datasets. Limitations primarily arise from project-specific edge cases, which may require additional rules but do not compromise the overall structure of the pipeline.

5.4 Downstream Task: Question Answering

To further validate the practical utility of the normalized dataset, a downstream task based on question answering (QA) was implemented. The goal of this task is not to propose a new QA model or framework, but to demonstrate that the structured representation produced by the normalization pipeline enables reliable analytical querying over real-world permitting data.

5.4.1 Task Setup

The QA task operates on the normalized long-format dataset stored in CSV format. Each row represents a single comment–response pair augmented with structured metadata such as project identifier, agency, and review round.

A question answering interface is implemented using a dataframe-centric approach, where natural language questions are translated into executable data operations over the normalized table. This design directly leverages the benefits of schema normalization: once all source files are transformed into a unified structure, the same class of questions can be applied consistently across projects and agencies.

To limit scope and ensure interpretability, the QA task focuses on factual and aggregative queries, such as counts, averages, maxima, and filtering based on structured attributes.

All QA experiments in this thesis are performed on the normalized outputs produced under the *one-description* schema matching setting, for both the training and test splits. This choice is motivated by the schema matching results reported in Section 6.3, where the one-description setting yields more reliable normalization quality than the three-description voting variant.

5.4.2 Query Execution via Code Generation

Questions are answered using a dataframe-centric interface in which a language model generates executable Python code over the normalized dataframe. Concretely, the implementation uses **PandasAI** [30] to translate each natural language question into a sequence of pandas operations (e.g., filtering, grouping, aggregation), execute the generated code against the dataframe, and return the resulting value.

The workflow consists of the following stages:

1. **Data preparation.** The normalized comments CSV file is loaded into memory as a pandas dataframe. Date-like columns are parsed into `datetime` objects, and derived metrics required by the test suite (e.g., response time in days) are computed to support response-time queries.
2. **LLM integration.** A custom connector interfaces PandasAI with a local Ollama endpoint, handling prompt construction, API communi-

cation, and response parsing.

3. **Agent initialization.** A PandasAI **Agent** is instantiated by combining the dataframe with the LLM connector.
4. **Question processing.** For each question, the agent prompts the model to generate Python code that answers the query using pandas operations. The generated code is then executed over the dataframe and the resulting value is returned.

For example, asking “What is the most common agency?” typically triggers code that computes a frequency table over the `agency` column (e.g., via `value_counts()`) and returns the most frequent entry.

5.4.3 Model Selection

Two language models were evaluated for the QA task:

- `deepseek-coder:6.7b`
- `qwen2.5-coder:7b`

Both models were used to generate executable code in response to natural language queries. Empirical testing showed that `qwen2.5-coder:7b` produced more reliable and complete code generation across a wide range of queries. In contrast, `deepseek-coder:6.7b` occasionally failed to generate valid or complete code segments, resulting in execution errors for otherwise well-formed questions.

Due to these stability issues, `qwen2.5-coder:7b` was selected as the final model for the QA task. The selection was based on its significantly higher success rate in generating executable and correct code segments compared to `deepseek-coder:6.7b`. Specifically, in comparative testing across the evaluation suite, `qwen2.5-coder:7b` achieved a pass rate of up to 60%, whereas `deepseek-coder:6.7b` frequently failed due to execution errors or incomplete code snippets, succeeding in fewer than 27% of cases. This performance gap is detailed further in Section 6.6.

Due to these stability issues, `qwen2.5-coder:7b` was selected as the final model for the QA task. The selection was based on qualitative reliability rather than benchmark accuracy, reflecting the exploratory and validation-oriented role of QA in this thesis. Both models were tested under the same QA test suite and configuration, and the selection reflects observed reliability

in producing executable code rather than a claim of superior general-purpose language understanding.

5.4.4 Evaluation Strategy

The QA system was evaluated using a predefined set of test questions designed to probe different aspects of the normalized data. The suite covers basic grouped statistics, response-time summaries, and cross-group comparisons, and spans three difficulty tiers (easy/medium/hard). Overall, the test suite contains 15 questions; the complete list and QA outcomes are reported in Section 6.6.

For each question, the generated answer is compared against a reference value computed explicitly using standard pandas operations. Answers are considered correct if they match the reference value exactly, or for numeric outputs, if they agree within a small tolerance.

The verification logic supports multiple comparison modes. For numeric outputs, the system attempts to extract numeric values from both the model-produced answer and the reference value and checks agreement within tolerance. For categorical outputs, the system normalizes strings and checks for exact matches. In addition, a soft-matching rule is used to handle cases where the model returns a full-sentence response (e.g., “The most common agency is EMLI”) while the reference answer is a short token (e.g., `EMLI`).

5.4.5 Scope and Limitations

The QA task is intentionally constrained to structured and semi-structured queries that can be resolved through deterministic dataframe operations. More open-ended or interpretive questions are outside the scope of this evaluation.

Importantly, the QA component is strictly downstream of the normalization pipeline and does not influence schema matching or value-level processing. Its primary role is to demonstrate that the normalized dataset supports reliable analytical access and enables automated querying that would not be feasible using the original heterogeneous spreadsheets.

Chapter 6

Experiments and results

6.1 Overview of Evaluation Setup

This chapter presents the experimental results of the proposed normalization pipeline, with a primary focus on schema matching. Schema matching constitutes the core component of the pipeline, as its output directly determines the structural correctness of the normalized data and strongly influences the behavior of downstream value matching and question answering components.

The evaluation is conducted on both training and test datasets, each examined under two experimental settings:

- Using one generated description per column.
- Using three generated descriptions per column.

Consistent with the definitions in Chapter 4, the training set is used for pipeline development and rule refinement, while the test set serves as a held-out collection to evaluate the pipeline’s ability to generalize to unseen spreadsheet structures.

In the three-description setting, multiple schema matching outputs produced by Jellyfish are aggregated through a majority voting mechanism to obtain a final mapping decision. All other components of the pipeline remain identical across settings.

Due to the absence of ground-truth labels and the real-world nature of the data, all evaluations in this chapter are performed manually. For each project, the output of the schema matching stage is inspected at both the column level and the structural level to determine whether the generated normalized representation is usable for downstream analysis. The evaluation criteria reported in the result tables are defined as follows.

Correct Sheet. Some projects contain multiple sheets, including meta-data sheets or multiple permit decision processes. A project is considered to have the correct sheet detected if the selected sheet corresponds to the main permit process conversation. When multiple permit process sheets exist, the sheet with the largest number of conversational rows is considered correct. Sheet selection is performed using rule-based heuristics.

Correct Sheet. Some projects contain multiple sheets, including meta-data sheets or multiple permit decision processes. A project is considered to have the correct sheet detected if the selected sheet corresponds to the main permit process conversation. When multiple permit process sheets exist, the sheet with the largest number of conversational rows is considered correct. For the purpose of this evaluation, the ground truth for the “correct” sheet was manually cross-checked to ensure that the selected worksheet contained the primary comment-tracking records rather than auxiliary data or reference lists. The selection of sheets in the pipeline is performed using rule-based heuristics.

Correct Column Header. Sheets may contain multiple header rows or use initial rows for metadata. Correct column header detection indicates that the pipeline successfully identified the row corresponding to the actual column headers used in the permit process data.

Useful Columns. Not all columns present in a project contribute to the normalized target schema. Useful columns are defined as source columns that semantically correspond to one of the target schema fields or are required for structural detection, such as comment or response text, author, agency, or status. Project-specific or auxiliary columns that do not map to the target schema are excluded from this count.

Column Detection Accuracy. Column detection accuracy is computed as the ratio of useful source columns that are correctly mapped to their corresponding target schema fields. This metric captures column-level performance but does not fully reflect the functional correctness of the normalized output.

Minor and Major Issues. Detected errors are categorized as either minor or major. Minor issues correspond to incorrect or missing mappings in auxiliary metadata fields, such as author, agency, topic, section, dates, or

status. Major issues correspond to incorrect or missing mappings in core conversational fields, including comment text, response text, round detection, or structural format (wide vs. long). Major issues directly affect the integrity of the conversational structure.

Final Result. Each project is assigned a final result label based on the overall usability of its normalized output.

- **Correct:** Projects with no issues or only minor issues (e.g., *2021 02 10 NewAfton_B3_SSN_ITT* in Table 6.1, which achieved 100% accuracy with no structural errors).
- **Semi-Correct:** Projects with correctly detected structure but with one or more major issues (e.g., *2021 09 23 QR Mine ITT* in Table 6.1, where the wide format was identified, but core conversational fields had mapping errors).
- **Incorrect:** Projects with incorrect structural detection are labeled as incorrect, regardless of column detection accuracy (e.g., *2021 06 13 BonanzaLedge ITT* in Table 6.1, where a structural misclassification rendered the output unusable despite 82.4% column matching accuracy).

Four main result tables are reported in this chapter, corresponding to the following configurations: (1) training set with one description, (2) training set with three descriptions, (3) test set with one description, and (4) test set with three descriptions. Each table reports detailed project-level outcomes based on the criteria defined above.

6.2 Evaluation Metrics and Their Limitations

Column detection accuracy is defined as the fraction of useful source columns that are correctly mapped to their corresponding target schema columns. While this metric provides a coarse quantitative signal of schema matching performance, it does not fully capture the functional correctness of the normalization output.

In wide-format projects with many useful columns, a high column detection accuracy can still co-exist with critical failures. For example, mis-detecting a single column corresponding to a later-round comment text may result in an incorrect structural interpretation of the entire project, leading to a fully unusable normalized representation.

Conversely, projects with lower column detection accuracy may still yield structurally correct outputs if the core conversational fields, such as comment text, response text, and round information, are correctly detected. In such cases, missing auxiliary metadata columns reduces completeness but does not invalidate the conversational structure.

For this reason, column detection accuracy is reported alongside a higher-level *final result* metric, which reflects whether the normalized output preserves the correct structure and conversational integrity of the project. This combined evaluation provides a more realistic assessment of schema matching performance in the absence of ground-truth labels. In practice, the relationship between these metrics is interpreted as follows:

- **High Column Accuracy and High Structural Correctness:** Indicates a fully successful normalization where both the conversational integrity and auxiliary metadata are preserved.
- **High Column Accuracy and Low Structural Correctness:** Signals a critical failure; despite matching most headers, the misidentification of a single structural anchor (like a round-specific column such as comment and response) renders the dataset conversationally invalid.
- **Low Column Accuracy and High Structural Correctness:** Represents a “semi-correct” or usable output where the core conversation is intact, though some metadata fields may be missing.
- **Low Column Accuracy and Low Structural Correctness:** Indicates a complete failure of the pipeline to identify either the schema or the structural layout of the spreadsheet.

6.3 Schema Matching Results

6.3.1 Training Set Results

One-Description Setting

Table 6.1 reports the schema matching results on the training set using a single generated description per source and target column. For each project, the table summarizes whether the correct sheet and column header were detected, the total number of columns and useful columns, the detected structural format (wide or long), and the number of useful columns correctly mapped to the target schema.

6.3. Schema Matching Results

The column detection accuracy is computed as the ratio of correctly detected useful columns to the total number of useful columns. In addition, detected issues are categorized as either minor or major, depending on whether they affect auxiliary metadata fields or the core conversational structure of the project. Based on these observations, a final result label is assigned to each project as correct, semi-correct, or incorrect.

This setting serves as the baseline configuration for assessing the behavior of Jellyfish-based schema matching under minimal descriptive input.

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2021 01 25 NewAfton_B3_ITT	yes	yes	17	16	wide	yes	15	minor	correct	0.938
2021 02 10 NewAfton_B3_SSN_ITT	yes	yes	7	4	long	yes	4	-	correct	1
2021 06 13 Bonanza-Ledge_ITT	yes	yes	24	17	wide	no	14	both	incorrect	0.824
2021 07 21 Elk Gold_ITT	yes	yes	38	17	wide	yes	13	both	correct	0.765
2021 09 20 Brule_ITT	yes	no	24	14	wide	no	10	major	incorrect	0.714
2021 09 23 QR Mine_ITT	yes	yes	21	17	wide	yes	15	both	semi-correct	0.882
2021 12 01 Premier_ITT	yes	yes	19	12	wide	yes	10	minor	correct	0.833
2022 08 16 NewAfton_CZone_ITT	yes	yes	19	12	wide	yes	10	both	semi-correct	0.833
2022 12 23 Blackwater_ITT	yes	yes	16	11	wide	yes	10	minor	correct	0.909
2023 08 31 GH0 Phase 7-2 Project_ITT	yes	yes	16	10	long	yes	10	-	correct	1
2023 11 10 LCO ERX_CCR Phase2_ITT	yes	yes	18	9	long	yes	9	-	correct	1
2024 05 15 CGP_ITT	yes	yes	29	24	wide	yes	22	both	semi-correct	0.917
2025 01 03 Eskay BTS_ITT	yes	yes	32	26	wide	yes	22	minor	correct	0.846

Table 6.1: Per-project evaluation results on the training set using one description per source and target column.

Three-Description Setting

Table 6.2 presents the training set results obtained using three generated descriptions for sources, per selected source–target column pair. In this setting, embedding-based similarity is first used to preselect candidate column pairs. For each selected pair, three independent descriptions are generated for the source column, and Jellyfish is applied separately to each description.

The final schema matching decision is determined through majority voting across the three Jellyfish outputs. All other components of the pipeline, including sheet detection and column header detection, remain identical to the one-description setting.

This configuration is designed to evaluate whether increasing descriptive diversity improves the robustness and consistency of schema matching decisions.

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2021 01 25 NewAfton_B3_ITT	yes	yes	17	16	wide	no	9	both	incorrect	0.562
2021 02 10 NewAfton_B3_SSN_ITT	yes	yes	7	4	long	yes	3	major	semi-correct	0.75
2021 06 13 Bonanza- Ledge_ITT	yes	yes	24	17	wide	no	13	major	incorrect	0.765
2021 07 21 Elk Gold_ITT	yes	yes	38	17	wide	no	9	both	incorrect	0.529
2021 09 20 Brule_ITT	yes	no	24	14	wide	no	9	major	incorrect	0.643
2021 09 23 QR Mine_ITT	yes	yes	21	17	wide	no	11	major	incorrect	0.647
2021 12 01 Premier_ITT	yes	yes	19	12	wide	no	8	both	incorrect	0.667
2022 08 16 NewAfton_CZone_ITT	yes	yes	19	12	wide	no	8	both	incorrect	0.667
2022 12 23 Blackwa- ter_ITT	yes	yes	16	11	wide	no	6	major	incorrect	0.545
2023 08 31 GH0 Phase 7-2 Project_ITT	yes	yes	16	10	long	yes	9	major	semi-correct	0.9
2023 11 10 LCO ERX_CCR Phase2_ITT	yes	yes	18	9	long	yes	7	major	semi-correct	0.778
2024 05 15 CGP_ITT	yes	yes	29	24	wide	no	17	both	incorrect	0.708
2025 01 03 Eskay BTS_ITT	yes	yes	32	26	wide	no	17	both	incorrect	0.654

Table 6.2: Per-project evaluation results on the training set using three descriptions per selected source–target candidate pairs with majority voting.

6.3.2 Test Set Results

One-Description Setting

Table 6.3 reports the schema matching results on the test set using a single description per source column. The test set consists of previously unseen projects and includes new column naming conventions, agency representations, and round encodings that may not appear in the training data.

The reported metrics and qualitative annotations follow the same evaluation protocol as the training set. This setting evaluates the generalization behavior of the schema matching pipeline under minimal descriptive input.

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2020_M-229_Thickener_ITT	yes	yes	20	17	wide	yes	13	both	semi-correct	0.765
2020_M-243_WestZoneRamp_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2021_M-11_HHDam_ITT	yes	yes	13	12	wide	yes	10	minor	correct	0.833
2021_M-173_TSF But-tress_ITT	yes	yes	11	11	long	yes	8	minor	correct	0.727
2021_M-229_ElkGold_ITT	yes	yes	20	16	wide	yes	12	minor	correct	0.75
2022_M-1_Silvertip_ITT	yes	yes	14	10	long	yes	9	-	correct	0.9
2022_M-11_Raven_ITT	yes	yes	18	15	wide	yes	10	minor	correct	0.667
2022_M-117_Blackwater_ITT	yes	yes	22	12	wide	yes	9	minor	correct	0.75
2022_M-12_NewAfton_CZone_ITT	yes	yes	17	12	wide	yes	11	-	correct	0.917
2022_M-137_GaloreCreek_ITT	yes	yes	13	12	wide	yes	11	-	correct	0.917
2022_M-17_GHO Phase 8_ITT	yes	yes	21	13	long	yes	13	-	correct	1
2022_M-19_KSM_ITT	yes	yes	16	12	wide	no	10	major	incorrect	0.833
2022_M-26_RedChris_ITT	yes	yes	12	11	long	yes	8	minor	correct	0.727
2022_M-32_MountMilligan_ITT	yes	yes	11	9	long	yes	9	-	correct	1
2022_M-33_GHO Phase 7-2 Project_ITT	yes	yes	19	10	long	yes	10	-	correct	1
2022_M-35_Silvertip_CCR_ITT	yes	yes	17	11	wide	yes	8	major	semi-correct	0.727

Continued on next page

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2022_M-4_GHO Phase 7-2 Project_ITT	yes	yes	17	11	wide	yes	10	-	correct	0.909
2022_M-46_Blackwater_ITT	yes	yes	26	13	long	yes	11	minor	correct	0.846
2023_M-10_BonanzaLedge_ITT	no	yes	7	7	long	yes	6	-	correct	0.857
2023_M-130_KSM_ITT	yes	yes	11	9	long	yes	9	-	correct	1
2023_M-162_GHO Phase 7-2 Project_ITT	yes	yes	21	13	long	yes	13	-	correct	1
2023_M-165_BonanzaLedge_ITT	yes	yes	17	12	wide	yes	10	-	correct	0.833
2023_M-172_KSM_ITT	yes	yes	16	11	long	yes	10	-	correct	0.909
2023_M-186_GHO Phase 7-2 Project_ITT	yes	yes	15	12	wide	yes	10	-	correct	0.833
2023_M-187_Raven_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2023_M-19_KSM_CCR_ITT	yes	yes	18	11	wide	yes	7	major	semi-correct	0.636
2023_M-20_Silvertip_CCR_ITT	yes	yes	15	10	long	yes	9	-	correct	0.9
2023_M-207_KSM_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2024_M-115_KSM_ITT	yes	yes	13	11	wide	no	7	major	incorrect	0.636
2024_M-148_GaloreCreek_ITT	yes	yes	18	12	wide	yes	10	-	correct	0.833
2024_M-172_KSM_ITT	yes	yes	12	9	long	yes	8	minor	correct	0.889
2024_M-19_KSM_ITS_ITT	yes	yes	18	12	wide	yes	8	-	correct	0.667
2024_M-200_KSM_ITT	yes	yes	12	12	wide	yes	10	minor	correct	0.833
2024_M-229_ElkGold_ITT	yes	yes	19	13	wide	yes	10	minor	correct	0.769

Continued on next page

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2024_M-243_WestZoneRamp_ITT	yes	yes	12	11	long	yes	10	-	correct	0.909
2025_M-243_ProductionRate_ITT	yes	yes	20	12	long	yes	11	-	correct	0.917
2025_M-40_SaddleDam_ITT	yes	yes	15	11	long	yes	11	-	correct	1

Table 6.3: Per-project evaluation results on the test set using one description per source and target column.

Three-Description Setting

Table 6.4 presents the schema matching results on the test set using three generated descriptions per source column and majority voting. This setting mirrors the three-description configuration used for the training set and evaluates whether increased descriptive diversity improves robustness when applied to unseen projects.

All evaluation criteria remain unchanged to ensure comparability across configurations.

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2020_M-229_Thickener_ITT	yes	yes	20	17	wide	no	13	both	incorrect	0.765
2020_M-243_WestZoneRamp_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2021_M-11_HHDam_ITT	yes	yes	13	12	wide	no	10	both	incorrect	0.833
2021_M-173_TSF But-tress_ITT	yes	yes	11	11	long	yes	9	minor	correct	0.818
2021_M-229_ElkGold_ITT	yes	yes	20	16	wide	yes	13	minor	correct	0.812
2022_M-1_Silvertip_ITT	yes	yes	14	10	long	yes	9	-	correct	0.9
2022_M-11_Raven_ITT	yes	yes	18	15	wide	yes	11	minor	correct	0.733
2022_M-117_Blackwater_ITT	yes	yes	22	12	wide	no	9	both	incorrect	0.75
2022_M-12_NewAfton_CZone_ITT	yes	yes	17	12	wide	yes	11	-	correct	0.917
2022_M-137_GaloreCreek_ITT	yes	yes	13	12	wide	yes	11	-	correct	0.917
2022_M-17_GHO Phase 8_ITT	yes	yes	21	13	long	yes	13	-	correct	1
2022_M-19_KSM_ITT	yes	yes	16	12	wide	no	9	major	incorrect	0.75
2022_M-26_RedChris_ITT	yes	yes	12	11	long	yes	9	minor	correct	0.818
2022_M-32_MountMilligan_ITT	yes	yes	11	9	long	yes	9	-	correct	1
2022_M-33_GHO Phase 7-2 Project_ITT	yes	yes	19	10	long	yes	10	-	correct	1
2022_M-35_Silvertip_CCR_ITT	yes	yes	17	11	wide	yes	10	minor	correct	0.909

Continued on next page

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2022_M-4_GHO Phase 7-2 Project_ITT	yes	yes	17	11	wide	yes	10	-	correct	0.909
2022_M-46_Blackwater_ITT	yes	yes	26	13	long	yes	11	minor	correct	0.846
2023_M-10_BonanzaLedge_ITT	no	yes	7	7	long	yes	6	-	correct	0.857
2023_M-130_KSM_ITT	yes	yes	11	9	long	yes	9	-	correct	1
2023_M-162_GHO Phase 7-2 Project_ITT	yes	yes	21	13	long	yes	13	-	correct	1
2023_M-165_BonanzaLedge_ITT	yes	yes	17	12	wide	yes	10	-	correct	0.833
2023_M-172_KSM_ITT	yes	yes	16	11	long	yes	10	-	correct	0.909
2023_M-186_GHO Phase 7-2 Project_ITT	yes	yes	15	12	wide	yes	10	-	correct	0.833
2023_M-187_Raven_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2023_M-19_KSM_CCR_ITT	yes	yes	18	11	wide	yes	8	major	semi-correct	0.727
2023_M-20_Silvertip_CCR_ITT	yes	yes	15	10	long	yes	9	-	correct	0.9
2023_M-207_KSM_ITT	yes	yes	11	11	long	yes	10	minor	correct	0.909
2024_M-115_KSM_ITT	yes	yes	13	11	wide	no	8	major	incorrect	0.727
2024_M-148_GaloreCreek_ITT	yes	yes	18	12	wide	yes	10	-	correct	0.833
2024_M-172_KSM_ITT	yes	yes	12	9	long	yes	8	minor	correct	0.889
2024_M-19_KSM_ITS_ITT	yes	yes	18	12	wide	yes	8	-	correct	0.667
2024_M-200_KSM_ITT	yes	yes	12	12	wide	yes	10	minor	correct	0.833
2024_M-229_ElkGold_ITT	yes	yes	19	13	wide	yes	11	minor	correct	0.846

Continued on next page

Project	Sheet	Header	#Cols	#Used	Fmt	Fmt OK	#Correct	Issue	Final	Acc.
2024_M- 243_WestZoneRamp_ITT	yes	yes	12	11	long	yes	10	-	correct	0.909
2025_M- 243_ProductionRate_ITT	yes	yes	20	12	long	yes	11	-	correct	0.917
2025_M- 40_SaddleDam_ITT	yes	yes	15	11	long	yes	11	-	correct	1

Table 6.4: Per-project evaluation results on the test set using three descriptions per selected source–target candidate pairs with majority voting.

6.4 Aggregated Project-Level Results

This section presents aggregate schema matching results at the project level. Beyond column-level correctness, correctly detecting the structural format of a project as either wide or long is essential for producing usable normalized outputs. An incorrect structural decision alters the interpretation of conversational rounds and propagates errors to downstream stages.

In this evaluation, structural detection errors are treated as major issues. In wide-format projects, incorrect detection may collapse multiple conversational rounds into a single long structure, whereas in long-format projects, misclassification may fragment a single conversational thread into multiple unrelated records. Both cases result in a loss of conversational integrity that cannot be recovered through value matching.

Structural correctness is therefore incorporated directly into the final project-level assessment. Projects with no issues or only minor issues are labeled as correct. Projects with correctly detected structure but with one or more major issues are labeled as semi-correct. Projects with incorrect structural detection are labeled as incorrect, regardless of column detection accuracy.

Table 6.5 summarizes the distribution of final result classifications across training and test datasets. This table aggregates column-level correctness, structural detection, and issue severity into a single project-level outcome.

Dataset	Setting	Correct	Semi-Correct	Incorrect
Train	1-Desc	8 (61.5%)	2 (15.4%)	3 (23.1%)
Train	3-Desc	0 (0.0%)	9 (69.2%)	4 (30.8%)
Test	1-Desc	20 (57.1%)	4 (11.4%)	11 (31.4%)
Test	3-Desc	17 (48.6%)	6 (17.1%)	12 (34.3%)

Table 6.5: Final Result Correctness across Training and Test sets.

To complement the final result classification, Table 6.6 reports aggregate accuracy metrics across datasets and experimental settings. In addition to average column detection accuracy, the table reports the average structure detection accuracy (**Avg Struct. Acc.**), defined as the fraction of projects for which the wide/long format is detected correctly. This value is computed by averaging the binary correctness of structural detection across all projects within each dataset and setting.

6.5. Value Matching: Qualitative Error Analysis

Dataset	Setting	Avg Col. Acc.	Avg Struct. Acc.
Train	1-Desc	0.879	0.846
Train	3-Desc	0.725	0.308
Test	1-Desc	0.853	0.886
Test	3-Desc	0.864	0.829

Table 6.6: Average Column Detection and Structure Detection Accuracy.

Together, these results show that column detection accuracy alone is insufficient to characterize schema matching performance. While higher column accuracy is generally desirable, failures in structural detection can dominate the final outcome by producing conversationally invalid normalized representations.

6.5 Value Matching: Qualitative Error Analysis

Evaluating the value matching component presents additional challenges due to its rule-based design and strong dependency on upstream schema matching decisions. Unlike schema matching, value matching does not lend itself to a single quantitative metric, particularly in the absence of ground-truth labels.

To assess value matching behavior, a qualitative evaluation strategy is adopted. For each project, three random samples of mapped `comment_id` entries are manually inspected. This sampling strategy is sufficient in practice, as the internal structure of a project is typically consistent across rows, and systematic errors become evident even with a small number of samples.

Several categories of value matching errors are observed in the test set. First, when the agency column is not detected during schema matching, the pipeline attempts to extract agency information from the author column. This fallback mechanism fails in cases where agency names are represented in previously unseen formats, which were not observed during training and are therefore not covered by the existing value mapping rules.

Second, incorrect sheet detection occasionally leads to value matching errors. In projects containing multiple sheets with permit process conversations, the pipeline selects the sheet with the largest number of rows. This heuristic fails in some cases due to hidden or empty rows that inflate the apparent

size of non-relevant sheets.

Third, value matching errors arise from unseen agency abbreviations in the test set. When new agency names appear that are not included in the abbreviation mapping derived from the training data, incorrect or missing agency normalization occurs.

Finally, round detection errors are observed in projects that use previously unseen round encoding patterns. For example, comment identifiers of the form 20.1, 20.2, and 20.3 represent multiple conversational rounds associated with a single comment identifier. Such patterns were not present in the training data and therefore were not captured by the round detection rules.

In many cases, observed value matching errors are not intrinsic failures of the value mapping rules themselves, but rather the result of error propagation from earlier stages, particularly schema matching and structural detection. This dependency highlights the importance of robust schema matching as a prerequisite for reliable value-level normalization.

6.6 Question Answering: Results and Error Analysis

This section evaluates the downstream question answering (QA) component on the normalized outputs produced by the pipeline. The goal of this evaluation is not to benchmark large language models in general, but to assess whether the target-format representation enables realistic analytical queries to be answered automatically using dataframe operations. QA is implemented using PandasAI[30], where a code-generating model produces pandas expressions that are executed against the normalized dataframe.

Evaluation protocol. A fixed test suite of 15 analytical questions is used for all runs. The questions are grouped into three difficulty levels (easy/medium/hard) based on the required operations (single aggregation, multi-condition filtering and grouping, and multi-step reasoning across groups or time windows). For each question, a reference answer is computed using deterministic pandas code. Answers are scored as correct when the parsed PandasAI answer matches the reference answer (exact match for categorical outputs and numeric equality within the evaluation tolerance for floating-point results).

Models and datasets. Two code-oriented models are evaluated under the same PandasAI configuration: `qwen2.5-coder:7b` and `deepseek-coder:6.7b`. Each model is tested on both the training and test splits of the normalized dataset produced in the one-description setting. Table 6.7 summarizes the pass rate and runtime statistics.

Model	Split	Total	Passed	Failed	Success (%)	Avg Time (s)
qwen2.5-coder:7b	Train	15	9	6	60.0	8.50
qwen2.5-coder:7b	Test	15	8	7	53.3	8.40
deepseek-coder:6.7b	Train	15	4	11	26.7	13.45
deepseek-coder:6.7b	Test	15	2	13	13.3	9.61

Table 6.7: QA accuracy and runtime summary on the normalized dataframe using PandasAI. Average and median runtime are measured per question.

To provide a fine-grained view of model behavior beyond aggregate accuracy, Table 6.8 and Table 6.9 report the raw outputs produced by each QA model for every question in the test suite, alongside the corresponding reference answers.

ID	Question	Reference	Qwen (Train)	Qwen (Test)
1	How many comments are in round 1?	6659 / 5442	6659	5442
2	What is the most common agency?	EMLI	The most common agency is EMLI with 3781 occurrences.	The most common agency is EMLI.
3	Which round has the highest number of comments?	1	The round with the highest number of comments is 1.0 with 6659 comments.	The round with the highest number of comments is 1.0.
4	How many comments are from the most common agency?	3781 / 1882	3781	1882
5	How many unanswered comments are there (missing <code>date_responded</code>)?	6237 / 2595	6061	2582
6	What is the median response time in days for responded comments?	24.0 / 23.0	24.0	23.0
7	Which agency has the lowest median response time (days)?	LGL / HRFN	The agency with the lowest median response time is LGL with a median of 1.0 days.	<code>ERROR: import dateutil not in whitelist</code>
8	What percentage of responded comments were answered within 30 days?	65.51 / 75.9	36.12	79.27
9	In round 1, which agency has the most comments?	EMLI	The agency with the most comments in round 1 is EMLI.	The agency with the most comments in round 1 is EMLI.
10	Which agency has the highest unanswered rate (share of rows with missing <code>date_responded</code>)?	AGMP / E	The agency with the highest unanswered rate is <code>nan</code> with a rate of 43.93%.	The agency with the highest unanswered rate is EMLI with a rate of 28.33%.
11	Which agency has the highest 90th percentile response time (days)?	NLG / LBN	The agency with the highest 90th percentile response time is NLG.	<code>ERROR: name 'sns' is not defined</code>
12	How many comments received in the last 90 days are still unanswered?	8 / 0	0	0
13	Which project has the slowest median response time (days)?	2021_06_13_Bonanza-Ledge.ITT / 2023_M-112_RCP.ITT	<code>/.../temp-chart.png</code>	<code>ERROR: No code found</code>

Continued on next page

ID	Question	Reference	Qwen (Train)	Qwen (Test)
14	Which agency’s mean response time increases the most from round 1 to round 3+?	RDKS / EMLI	The agency with the highest increase in mean response time from round 1 to round 3+ is RDKS with an increase of 9.00 days.	The agency with the most significant increase in mean response time from round 1 to round 3+ is nan .
15	Which (project, agency) pair has the highest share of comments within that project?	(2022_08_16 NewAfton_CZone_ITT, EMLI) / (2021_M-173-TSF Buttress_ITT, EMLI)	ERROR: No code found	2024_M-11_MLE_ITT has 50.07%

Table 6.8: Per-question QA outputs for `qwen2.5-coder:7b`. “Reference” lists the deterministic pandas answer for train/test (when they differ). “ERROR” denotes execution failures or missing code generation.

ID	Question	Reference	DeepSeek (Train)	DeepSeek (Test)
1	How many comments are in round 1?	6659 / 5442	6659	5442
2	What is the most common agency?	EMLI	ERROR: import collections not in whitelist	ERROR: import collections not in whitelist
3	Which round has the highest number of comments?	1	/.../temp_chart.png	5442
4	How many comments are from the most common agency?	3781 / 1882	/.../temp_chart.png	ERROR: No code found
5	How many unanswered comments are there (missing <code>date_responded</code>)?	6237 / 2595	6061	/.../temp_chart.png
6	What is the median response time in days for responded comments?	24.0 / 23.0	ERROR: True	ERROR: No code found
7	Which agency has the lowest median response time (days)?	LGL / HRFN	ERROR: Unknown string format	The median response time in days for responded comments is nan.
8	What percentage of responded comments were answered within 30 days?	65.51 / 75.9	64.94534489257444	ERROR: import re not in whitelist
9	In round 1, which agency has the most comments?	EMLI	/.../temp_chart.png	/.../temp_chart.png
10	Which agency has the highest unanswered rate (share of rows with missing <code>date_responded</code>)?	AGMP / E	The agency with the highest unanswered rate is AGMP.	The agency with the highest unanswered rate is E.
11	Which agency has the highest 90th percentile response time (days)?	NLG / LBN	The agency with the highest 90th percentile response time is NLG.	ERROR: name 'ZeroDivisionError' is not defined
12	How many comments received in the last 90 days are still unanswered?	8 / 0	1018	6565
13	Which project has the slowest median response time (days)?	2021_06_13 Bonanza-Ledge.ITT / 2023_M-112.RCP.ITT	Table output (max=BonanzaLedge.ITT)	ERROR: No code found
14	Which agency's mean response time increases the most from round 1 to round 3+?	RDKS / EMLI	ERROR: No code found	ERROR: No code found

Continued on next page

ID	Question	Reference	DeepSeek (Train)	DeepSeek (Test)
15	Which (project, agency) pair has the highest share of comments within that project?	(2022-08-16 / .../temp-chart.png NewAfton_CZone_ITT, EMLI) / (2021_M- 173_TSF Buttress_ITT, EMLI)		ERROR: No code found

Table 6.9: Per-question QA outputs for `deepseek-coder:6.7b`. “Reference” lists the deterministic pandas answer for train/test (when they differ). “ERROR” denotes execution failures or missing code generation.

Failure modes. Manual inspection of the logged raw outputs indicates four recurring error categories. First, *execution failures* occur when generated code uses non-whitelisted imports (e.g., `collections`, `dateutil`, `re`) or references unavailable libraries (e.g., `sns`). Second, *non-scalar outputs* are produced for scalar questions, where PandasAI returns a plot artifact path (e.g., `temp_chart.png`) instead of a value. Third, *missing-value handling errors* lead to spurious NaN answers or incorrect grouping results (e.g., returning an agency value of `nan` as the maximum). Finally, *logic errors* arise from partially correct but incorrect filtering conditions, such as undercounting unanswered comments or miscomputing the share of responses within a time threshold.

These observations suggest that QA performance is currently limited by the robustness of code generation under strict execution constraints and by the sensitivity of some queries to null handling and multi-step aggregation logic. Nevertheless, the results demonstrate that the normalized target schema supports a broad class of analytical questions, and that a code-oriented model can answer a majority of the test suite correctly without additional feature engineering.

Overall performance. Across both splits, `qwen2.5-coder:7b` substantially outperforms `deepseek-coder:6.7b`, achieving 60.0% accuracy on the training split and 53.3% on the test split. In contrast, `deepseek-coder:6.7b` reaches 26.7% on training and 13.3% on test. The gap is largely due to execution reliability: `deepseek-coder` more often emits code that violates the PandasAI whitelist or fails to return an executable snippet.

Despite the observed limitations in automated question answering, the practical usefulness of the normalization pipeline remains significant for three reasons. First, the pipeline successfully transforms heterogeneous layouts into a unified structure that previously required hours of manual alignment. Second, the normalized CSV format allows human analysts to perform reliable, deterministic queries using standard tools without the risk of model-generated code errors. Finally, the majority of QA failures were attributed to the execution constraints of the local LLM rather than the integrity of the underlying data, suggesting that the integrated dataset is a robust foundation for future, more advanced analytical agents.

6.7 Summary of Key Findings

The experimental results lead to several key observations regarding the behavior and limitations of the proposed schema matching pipeline.

First, increasing the number of generated descriptions from one to three does not lead to consistent improvements in schema matching performance. On the training set, the three-description setting substantially degrades both column detection accuracy and structural detection performance. This degradation suggests that generating multiple description variants may introduce semantic noise or "hallucinated" context that conflicts with the voting mechanism, particularly in complex wide-format layouts where structural integrity depends on the precise matching of every round-specific column. On the test set, increasing the number of descriptions has a limited effect on the final outcomes, with most columns correctly detected in the one-description setting also being detected in the three-description setting. This discrepancy between the datasets may indicate that the test projects featured more distinct column headers or less ambiguous semantic roles, making them more resilient to the variations in LLM-generated descriptions. Furthermore, the results imply that the embedding-based prefiltering and Jellyfish inference reach a point of "diminishing returns" where additional descriptive diversity introduces more ambiguity than clarity.

Second, among all target fields, the **agency** column is the only case in which the three-description setting occasionally improves detection compared to the one-description baseline. For the majority of other columns, additional descriptions do not provide measurable benefits.

Third, schema matching performance is influenced by the availability and quality of sample values. Columns with identical or highly similar names and descriptions may be detected differently depending on whether they contain non-empty values. In particular, columns with empty or sparse values are more likely to be missed compared to semantically equivalent columns in other projects that include representative samples.

Another important observation concerns the role of column naming in schema matching performance. The proposed pipeline relies heavily on column names and their generated descriptions, making it sensitive to ambiguous or misleading naming conventions in real-world spreadsheets. In several projects, column names that intuitively suggest one semantic role are used to represent a different concept. For example, columns named **comment description** are expected to correspond to comment text, while in prac-

tice they are used to represent the comment type. When such columns are incorrectly detected as comment text, the resulting schema mapping leads to incorrect structural interpretation and round detection. A similar issue is observed for columns named `issue id`, which in some projects are used to represent comment identifiers rather than issue-level identifiers.

Finally, the results demonstrate that schema matching errors propagate to downstream components. Even when value mapping rules are correctly designed, mis-detection of schema elements or structural format can lead to incorrect value-level outputs. This dependency highlights the central role of schema matching in determining the overall reliability of the normalization pipeline.

6.8 Reflection on Strategic Objectives

Following the presentation of results, it is pertinent to reflect on the delivery of the core architectural goals: robustness, transparency, and generalizability.

- **Robustness and Model Agreement:** Robustness was assessed through the concordance of multiple LLMs. As evidenced in Table 5.2, while Mistral, Llama, and Neural-Chat reached a consensus on the semantic definition of temporal fields, their operational robustness varied. For instance, Mistral:7b demonstrated superior formatting discipline, whereas Qwen3:8b introduced internal reasoning tokens (`< think >`) that necessitated additional sanitization, potentially compromising pipeline stability.
- **Transparency:** The integration of Chain-of-Thought prompting directly supports auditability. By providing natural-language justifications for each binary decision, the pipeline moves beyond "black-box" similarity scores to a framework where a regulatory officer can verify the logic behind a specific schema alignment.
- **Generalizability:** The generalization capability was empirically validated by the performance on the held-out test set. Despite encountering novel naming conventions and project-level structural variants not present in the training subset, the pipeline maintained a structure detection accuracy of 88.6%. This confirms that the rule-based normalization, when guided by LLM-assisted semantic matching, effectively scales to unseen administrative datasets.

6.9 Open Questions and Unexpected Behaviors

Despite detailed manual inspection of the results, several observations remain unexplained.

First, in the three-description setting, the test set exhibits better overall performance than the training set. This behavior is counterintuitive, as the training data is expected to be more aligned with the generated descriptions and value mapping rules.

Second, generating three descriptions significantly degrades performance on the training set while having only a marginal effect on the test set. The underlying cause of this discrepancy is not clear.

These observations suggest complex interactions between description generation, embedding-based prefiltering, and Jellyfish inference behavior. Understanding these interactions requires further investigation and is left for future work.

Chapter 7

Conclusion and Future Work

This thesis addressed the challenge of integrating heterogeneous, real-world environmental assessment review spreadsheets into a unified and analyzable representation. In the mine permitting context studied in this work, review data are recorded as iterative comment–response exchanges across multiple agencies, projects, and rounds, but are stored in Excel files with inconsistent schemas, naming conventions, and structural layouts. These inconsistencies make systematic analysis, cross-project comparison, and downstream data-driven tools difficult to apply at scale.

To address this problem, the thesis proposed and implemented an end-to-end normalization pipeline that transforms project-specific spreadsheets into a single long-format target schema. The pipeline combines deterministic preprocessing and value-level transformations with LLM-assisted schema matching, enabling semantic alignment of columns without requiring labeled training data. Richer column descriptions were generated to provide contextual information beyond raw headers, and an embedding-based prefiltering step was used to restrict candidate matches before applying the Jellyfish schema matcher. After schema alignment, rule-based value-level processing standardized identifiers, agencies, dates, and review rounds, producing a consistent CSV dataset suitable for analysis.

Experimental results demonstrated that the proposed approach is effective in practice. Manual evaluation across training and test projects showed that a single-description schema matching setting consistently outperformed a three-description voting strategy, particularly for wide-format spreadsheets where structure detection is sensitive to missing or misaligned round-specific fields. The results also highlighted important distinctions between column-level accuracy and end-to-end correctness, motivating the use of qualitative final-result categories alongside quantitative metrics. Overall, the pipeline successfully normalized a diverse collection of real-world spreadsheets, enabling reliable reconstruction of multi-round review conversations across

projects.

As a downstream validation, the thesis evaluated question answering (QA) over the normalized dataset. Using a dataframe-based QA setup with local code-oriented language models, the experiments assessed whether the target-format representation supports realistic analytical queries. The evaluated questions covered common analysis tasks such as counting comments by agency or round, computing response-time statistics, and identifying unanswered or delayed responses. Although QA is not a primary contribution of this work, the results demonstrate that schema normalization substantially improves data accessibility and enables automated analytical querying with moderate reliability under strict execution constraints.

7.1 Limitations

Despite its effectiveness, the proposed approach has several limitations.

First, evaluation relied on manual inspection due to the absence of ground-truth labels. While this reflects the realities of real-world administrative data, it limits the ability to report fully objective performance metrics and makes large-scale automated evaluation infeasible.

Second, schema matching performance depends on the quality and representativeness of column values. Columns with sparse or empty values were more likely to be mis-detected, even when their semantic role was clear in other projects. Similarly, structurally critical fields (e.g., round-specific comment or response columns) have a disproportionate impact on end-to-end correctness that is not fully captured by column-level accuracy metrics.

Third, value-level processing is rule-based and therefore sensitive to previously unseen patterns. New agency naming conventions, alternative round encodings, or unexpected author formats can lead to errors, particularly in the test set. Although the deterministic design improves transparency and debuggability, it requires ongoing rule maintenance as new data are introduced.

Another important limitation of the proposed pipeline is error propagation across stages. The normalization process is sequential: description generation informs schema matching, schema matching constrains value-level processing, and the normalized output is subsequently used for downstream question answering. As a result, errors introduced at earlier stages can propagate and amplify in later stages. This dependency highlights the impor-

tance of robust description generation and early-stage validation, as downstream components have limited ability to recover from upstream semantic errors.

Finally, the downstream question answering experiments were limited in scope. Only a subset of queries was tested, and QA performance was evaluated qualitatively rather than through a formal benchmark. As a result, conclusions about QA accuracy and robustness remain preliminary.

7.2 Future Work

Several directions for future work emerge from this study.

A first direction is improving schema matching robustness for structurally critical fields. This may include weighting certain target fields more heavily during matching, incorporating lightweight structural cues alongside semantic similarity, or introducing confidence-aware postprocessing to flag high-risk mappings for review.

Second, value-level normalization could be extended with adaptive or hybrid approaches. While deterministic rules provide transparency, selectively incorporating learned or LLM-assisted value parsing could help handle previously unseen patterns, such as novel agency names or unconventional round encodings, while retaining auditability.

Third, description generation and matching strategies could be further explored. The observed degradation when using multiple descriptions suggests that additional descriptions may introduce noise rather than clarity. Future work could investigate more constrained description generation, confidence calibration, or explanation-based filtering to better exploit multiple descriptions without harming performance.

Finally, question answering over normalized regulatory data represents a promising application area. Future work could define a standardized query set, evaluate QA performance quantitatively, and explore user-facing interfaces that allow analysts or reviewers to interactively query large collections of review data. Such tools could support transparency, accountability, and efficiency in regulatory review processes beyond the specific case study examined in this thesis.

In summary, this thesis demonstrates that combining LLM-assisted schema matching with deterministic normalization offers a practical and effective so-

7.2. *Future Work*

lution for integrating heterogeneous environmental assessment review data. The proposed pipeline bridges the gap between unstructured operational spreadsheets and structured analytical datasets, laying the groundwork for scalable analysis and intelligent downstream applications in regulatory contexts.

Bibliography

- [1] BC Environmental Assessment Office. Environmental assessment process overview, 2018. Online; accessed 2-Feb-2026. 1
- [2] Erhard Rahm and Philip A. Bernstein. A survey of approaches to automatic schema matching. *The VLDB Journal*, 10(4):334–350, December 2001. 10, 15, 21
- [3] Jing Zhang, Bonggun Shin, Jinho D Choi, and Joyce C Ho. Smat: An attention-based deep learning solution to the automation of schema matching. In *European Conference on Advances in Databases and Information Systems*, pages 260–274. Springer, 2021. 10, 11
- [4] Hong-Hai Do and Erhard Rahm. *Chapter 53 - COMA — A system for flexible combination of schema matching approaches*, page 610–621. Morgan Kaufmann, San Francisco, 2002. 10, 21
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. 11
- [6] Mingyu Jeon, Jaeyoung Suh, and Suwan Cho. Schema matching on graph: Iterative graph exploration for efficient and explainable data integration. (arXiv:2511.20285), December 2025. arXiv:2511.20285 [cs]. 11
- [7] Yunjia Zhang, Avrilia Floratou, Joyce Cahoon, Subru Krishnan, Andreas C. Müller, Dalitso Banda, Fotis Psallidas, and Jignesh M. Patel. Schema matching using pre-trained language models. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, page 1558–1571, April 2023. 11, 12, 15, 16
- [8] Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. Magneto: Combining small and large language mod-

- els for schema matching. *Proceedings of the VLDB Endowment*, 18(8):2681–2694, April 2025. 11, 15
- [9] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. Jellyfish: Instruction-tuning local large language models for data pre-processing. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8754–8782, Miami, Florida, USA, November 2024. Association for Computational Linguistics. 11, 12, 13
- [10] Eitam Sheetrit, Menachem Brief, Moshik Mishaeli, and Oren Elisha. Rematch: Retrieval enhanced schema matching with llms. (arXiv:2403.01567), May 2024. arXiv:2403.01567 [cs]. 11, 12
- [11] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. (arXiv:2310.06825), October 2023. arXiv:2310.06825 [cs]. 13, 18
- [12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The llama 3 herd of models. (arXiv:2407.21783), November 2024. arXiv:2407.21783 [cs]. 13, 18
- [13] Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. Platypus: Quick, cheap, and powerful refinement of llms. 2023. 13
- [14] Leonard Traeger, Andreas Behrend, and George Karabatis. Collaborative scoping: Self-supervised linkability assessment for schema matching, 2026. 14
- [15] Micha  l Patryk Miazga, Daniel Abitz, Matthias T  schner, and Erhard Rahm. Automated configuration of schema matching tools: A reinforcement learning approach. In *Proceedings of the 21st International Conference on Datenbanksysteme f  r Business, Technologie und Web (BTW 2025)*, page 331–354. Gesellschaft f  r Informatik, Bonn, 2025. 14
- [16] Ling Ling Yan, Ren  e J. Miller, Laura M. Haas, and Ronald Fagin. Data-driven understanding and refinement of schema mappings. In *Proceedings of the 2001 ACM SIGMOD international conference on*

- Management of data*, SIGMOD '01, page 485–496, New York, NY, USA, May 2001. Association for Computing Machinery. 15
- [17] Xiaoyong Chai, Mayssam Sayyadian, AnHai Doan, Arnon Rosenthal, and Len Seligman. Analyzing and revising data integration schemas to improve their matchability. *Proc. VLDB Endow.*, 1(1):773–784, August 2008. 15
- [18] Sha Wang, Yuchen Li, Hanhua Xiao, Bing Tian Dai, Roy Ka-Wei Lee, Yanfei Dong, and Lambert Deng. Llmatch: A unified schema matching framework with large language models. (arXiv:2507.10897), 2025. arXiv:2507.10897 [cs]. 15, 16
- [19] Mounica Maddela, Lingjue Xie, Daniel Preotiuc-Pietro, and Mausam. Starqa: A question answering dataset for complex analytical reasoning over structured databases. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, page 34487–34499, Suzhou, China, November 2025. Association for Computational Linguistics. 16
- [20] Cunxiang Wang, Ruoxi Ning, Boqi Pan, Tonghui Wu, Qipeng Guo, Cheng Deng, Guangsheng Bao, Xiangkun Hu, Zheng Zhang, Qian Wang, and Yue Zhang. Novelqa: Benchmarking question answering on documents exceeding 200k tokens, 2025. 16
- [21] Wei Zhou, Bolei Ma, Annemarie Friedrich, and Mohsen Mesgar. Table question answering in the era of large language models: A comprehensive survey of tasks, methods, and evaluation. (arXiv:2510.09671), October 2025. arXiv:2510.09671 [cs]. 16
- [22] LangChain. Benchmarking question/answering over csv data, August 2023. Online; accessed 10-Dec-2025. 16, 17
- [23] Haoyu Dong, Jianbo Zhao, Yuzhang Tian, Junyu Xiong, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, and Dongmei Zhang. Encoding spreadsheets for large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, page 20728–20748, Miami, Florida, USA, November 2024. Association for Computational Linguistics. 17

- [24] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. Tapex: Table pre-training via learning a neural sql executor. (arXiv:2107.07653), March 2022. arXiv:2107.07653 [cs]. 17
- [25] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. (arXiv:2210.02875), March 2023. arXiv:2210.02875 [cs]. 17
- [26] Kyosuke Nishida, Kugatsu Sadamitsu, Ryuichiro Higashinaka, and Yoshihiro Matsuo. Understanding the semantic structures of tables with a hybrid deep neural network architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1), February 2017. 17
- [27] Yiming Lin, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G. Parameswaran. Twix: Automatically reconstructing structured data from templated documents. (arXiv:2501.06659), January 2025. arXiv:2501.06659 [cs]. 17
- [28] Ollama. Online; accessed 22-Jan-2026. 18
- [29] Vikranth Udandaraao and Nipun Misra. Democratizing ai development: Local llm deployment for india’s developer ecosystem in the era of tokenized apis. (arXiv:2508.16684), August 2025. arXiv:2508.16684 [cs]. 18
- [30] Sinaptik AI. Pandasai: Conversational data analysis with generative ai. <https://github.com/sinaptik-ai/pandas-ai>, 2024. Online; accessed 4-Feb-2026. 48, 69

Appendix A

Representative Spreadsheet Heterogeneity

This appendix provides visual examples of the structural heterogeneity discussed in Section 4.1. To maintain data privacy, columns containing personal identifiers (e.g., specific author names) have been hidden.

A.1 Long-Format Example

	A	B	C	E	F	G	H	I	J	K	L	M	N	O
	Comment ID	Section	Topic	Comment Responder	Date Comment Received	Review Phase	New or Follow Up	Comment Type	Potential Permit Condition?	Reviewer's comment	Proponent's response	Date Response	Status	Follow Up
	Unique identifying ID to keep track of comments	Section of application or supporting document	Theme of comment	Name and/or organization	Date comment provided from reviewer	Round of review (1,2,3)	New/Follow Up	Information requirement, clarification, question, or comment	Yes/No	Reviewer's comment	Proponent's response	Date response provided by proponent	Open or closed (closure must be confirmed by reviewer)	Identify any follow-up actions or deliverables
56	36	Appendix 7-A AEMP Figure 4.2-1	Effluent		Jun 01, 2022	1	New			What is the rationale for having toxicity testing and temperature data collection at 661-10 in Creek 661 mainstem, rather than in upstream tributaries closer to sediment control ponds (e.g., 661-10)?	Please see the memo titled: Blackwater AEMP Comment Responses 39, 44, 45, 46, 47, 49, 51, 52, 53, 54, 56, 57, and 61 for additional information on updated monitoring sites in upper Creek 661.	30-Jun-22	Closed	
57	36	Appendix 7-A AEMP Figure 4.2-1	Effluent		Jul 22, 2022	2	Follow Up			1. Does 661-10 represent worst case water quality conditions (i.e., site where toxicity effects are most likely to occur)? 2. If not, where and when (time of year) would the worst-case water quality occur?	BW Gold will provide a response to comments related to the Camp Site SCP in a follow-up response on the date provided to the MMO in the most recent version of the Round 2 BW workplan.	25-Aug-22	Closed	
58	36	Appendix 7-A AEMP Figure 4.2-1	Effluent		Aug 25, 2022	3	Follow Up			Awaiting response from BW Gold	Please refer to response in memo titled: R2_ENV Comment IDs 36 and 88.	2-Sep-22	Closed	
59	37	Appendix 7-A AEMP Figure 4.2-1	Effluent		Jun 01, 2022	1	New			1. What is the rationale for having DC-15 (far field impact site) so far downstream (~dam downstream of near field)? 2. What additional mine effects are expected from DC-15?	1 and 2. Table 4.2-1 of the Aquatic Effects Monitoring Program (AEMP) Plan indicates that DC-15 is a near field impact site on Davidson Creek. The site, DC-05 (approximately 100 m downstream of the dam) is provided in response to comment 74 in Round 1, indicates that past activities have occurred throughout Davidson Creek. In upper Davidson Creek or areas closer to the mine site (e.g., DC-05, DC-10, DC-13), forestry activities along Davidson Creek have occurred in the past. Baseline studies have likely	30-Jun-22	closed	
60	37	Appendix 7-A AEMP Figure 4.2-1	Effluent		Jul 22, 2022	2	Follow Up			1. What is the furthest downstream location before non-mine activities would influence water quality? I'm interested in where an observed anthropogenic aquatic impact would likely be related to the mine and at what point it would become more uncertain due to	In upper Davidson Creek or areas closer to the mine site (e.g., DC-05, DC-10, DC-13), forestry activities along Davidson Creek have occurred in the past. Baseline studies have likely	25-Aug-22	closed	Thank-you for your response. Please also see my response to comment 40.

Figure A.1: Snapshot of project “2022 12 23 Blackwater_IT” illustrating a long-format structure. Note how the same Comment ID appears in multiple rows to represent different review rounds.

A.2. Wide-Format Example

A.2 Wide-Format Example

The following figures illustrate the horizontal growth of project “2022 08 16 NewAfton_CZone.IT”. Review rounds are recorded by appending groups of columns, necessitating extensive horizontal scrolling.

Comment ID	Document Title	Department	EMLI Technical Review Round 1 Comments	Comment Date	EMLI TO UPDATE Status (Open, Closed)
41	Technical Review Round 1 Re: New Afton M-229 C-Zone Permit Amendment Application EMLI Geotechnical Comments – Round 1 (2022-02-03 EMLI Geotech Comments New Afton-C-Zone MAPA round 1.pdf)	EMLI Geotechnical	even though we use all available geological data, the REV of available data under and around the TSFs is coarse for simulation of strain near those installations.” “Sub models that use the LR4-FS4 outputs should also include additional length scales of information, and not have downgraded resolution, e.g. be 2D, small strain, use stress as boundary conditions ignore intermediate stages or hydromechanical effects, and should especially not simplify the 3d geometry.” “The end user of these forecasts must therefore consider the resolution of data included in the model and appreciate that strain localisation on sub REV scale features is probable from time to time.” “If the mine wishes to achieve higher resolution estimates of	3-Feb-22	
42	Technical Review Round 1 Re: New Afton M-229 C-Zone Permit Amendment Application EMLI Geotechnical Comments – Round 1 (2022-02-03 EMLI Geotech Comments New Afton-C-Zone MAPA round 1.pdf)	EMLI Geotechnical	23. Please provide a list of proposed initial QPOs and TARP for the HATSF for C-Zone mining. The QPOs and TARP must include TSF stabilization and stabilization timing requirements. Please note some of these may be used to inform potential permit conditions. [EOR Response Required].	3-Feb-22	

Figure A.2: NewAfton C-Zone (Part 1 of 3): Initial columns and Round 1 metadata.

NAM Technical Review Round 1 Response	EMLI Technical Review Round 2 Comments	NAM Technical Review Round 2 Response
Refer to the attached: "EMLI Round 1 Comments-Responses (VA22-00246).pdf"	EMLI Round 2 Comment: Open pending receipt of report	[Information Requirement] acknowledged. KP's report is expected mid July. Refer to the file attached: "VA22-00789 - EMLI Round 2 Comments-Responses"
Refer to the attached: "EMLI Round 1 Comments-Responses (VA22-00246).pdf"	EMLI Round 2 Comment: The following are proposed draft permit conditions [Potential Permit Conditions] • The existing Permit M-229 conditions B.2.(b),(iii), and B.3.(f) (i) (ii) (iii) (iv) (v) will be updated to reflect C-Zone subsidence effects on the NATSF and HATSF. • The Proponent shall submit updated QPOs and TARP for the HATSF to the Chief Inspector prior to the C-Zone cave interacting with the B3-cave or extraction level	Potential permit condition acknowledged NAM is requesting more information on this perrr condition. As C-Zone mining subsidence progresses there be a permit condition for progressive stabiliza or will the facility need to be fully stabilized? Comment also provided: Refer to the file attached: "VA22-00789 - EMLI Round 2 Comments-Responses"

Figure A.3: NewAfton C-Zone (Part 2 of 3): Expansion into Round 2 comments and responses.

A.2. Wide-Format Example

	EMLI Technical Review Round 3 Comments	NAM Technical Review Round 3 Response	EMLI Technical Review Round 3B Comments	NAM Technical Review Round 3B Response	Open/Closed
41		FLAC Model update to be provided August 3rd, 2022 in a meeting held between NAM/SSN/Third Party/EMLI, Report to be provided by August 15, 2022			Open
42	EMLI Round 3 Comment: Closed [Potential Permit Conditions]				Closed, Possible Permit Condition

Figure A.4: NewAfton C-Zone (Part 3 of 3): Final columns including Round 3 and project status.

Appendix B

Prompt Templates Used in This Thesis

B.1 Description Generation Prompts

This section documents tableB.1 with the prompt templates used for source column description generation. All prompts are reported explicitly to ensure transparency and reproducibility. For readability, few-shot examples included in the original prompts are abbreviated using placeholders such as “[2 examples]” or “[5 examples]”.

P	FS	Key Goals	Prompt Highlights	Output
1	0	Max 25 words, Start with “This column”, Include data type, Explain purpose	Mining permit context. Maximum 25 words, Must start with “This column”, Include data type or format, Explain specific purpose in mining permit context.	Unable to distinguish comments/responses; too generic
2	2	Describe specific purpose based on context, Max 25 words, Focus on what it is, No justifications	Function: Describe column’s specific purpose based on Table Context. Infer entity from Column Name. Length: Strictly 25 words or less. Style: Be precise. Focus on what it is. [2 examples]	Best overall; minor comment/response confusion; used for final pipeline

Continued on next page

B.2 Jellyfish Schema Matching Prompts

Continued from previous page

P	FS	Key Goals	Prompt Highlights	Output
3	5	Build on P2: Be specific about direction, Never use “comment or response”, Distinguish REVIEWER to PROPONENT vs PROPONENT to REVIEWER	Be Specific About Direction: “comment”→REVIEWER to PROPONENT; “response”→PROPO- REVIEWER. NEVER use ambiguous phrases. Be Specific About Entity for text/date columns. [3 examples]	Overfitting; collapses to single template: “initial feedback or question from reviewer”
4	4	Build on P3: Identify column type first (meta-data/content/temporal)	Identify Column Type First: Metadata columns identify/categorize. Content columns be specific about direction. Temporal columns specific about event. NEVER ambiguous. [3 examples]	Same overfitting as P3; template collapse continues
5	2	P2 foundation: Add direction specification, Distinguish REVIEWER to PROPONENT vs PROPONENT to REVIEWER	Function: Describe specific purpose. Be Specific About Direction: “comment”→REVIEWER to PROPONENT; “response”→PROPO- REVIEWER. NEVER ambiguous. [2 examples]	Detected most comments as responses; direction alone insufficient

Table B.1: Prompting strategies for column description generation. P=Prompt, FS=Few Shots.

B.2 Jellyfish Schema Matching Prompts

Table B.2 reports the complete Jellyfish outputs for a diagnostic schema-matching example under different model sizes and prompting strategies. The evaluated source–target attribute pair is **Round 1: MDRC Member Comment** (source) and **response_text** (target), which is a known non-match. Full model responses are shown verbatim to document the effect of prompt design and reasoning style.

B.2. Jellyfish Schema Matching Prompts

Model	Prompt	Model output (verbatim)	Final answer
7B	No-CoT	Yes	Yes
7B	CoT	No, the two attributes are not semantically equivalent. While both attributes are related to the major mine project discussions, they serve different purposes. Attribute A records comments from MDRC members during Round 1 of the major mine project discussions, typically containing questions or clarifications related to the project documentation. On the other hand, Attribute B contains the proponent's written response to each comment made by a reviewer during the major mine project review process. Therefore, these attributes do not refer to the same information, as one attribute records the comments from MDRC members, and the other records the proponent's responses to those comments.	No
8B	No-CoT	Yes	Yes
8B	CoT	Yes. The two attributes, Round 1: MDRC Member Comment and response_text , are semantically equivalent. They both contain text-based feedback and questions related to various aspects of the project. The Round 1: MDRC Member Comment attribute records specific comments from MDRC members during Round 1 of the major mine project's review process, while the response_text attribute contains the proponent's written response to each comment made by a reviewer during the major mine project review process. Both attributes provide information about the project's aspects and are thus semantically equivalent. SELECT * FROM table1 WHERE attribute_a = attribute_b;	Yes

Table B.2: Complete Jellyfish outputs for schema matching under different model sizes (7B, 8B) and prompting strategies (with and without chain-of-thought). The evaluated source–target attribute pair is **Round 1: MDRC Member Comment** and **response_text**, for which the correct decision is **No**.